Complexitylib:
An index of proved statements

20 October 2026

This tour collects 126 entries spanning the library's significant results, from machine complexity and descriptive logic to cryptography, circuit lower bounds, proof complexity, communication, coding and circuit synthesis. Some entries group closely related theorems. It is a statement index, not an exhaustive API reference or a translation of proof bodies. Each entry identifies its hypotheses, model, quantitative guarantee and scope, with links pinned to reviewed library revisions: `8d38f775` for the base entries, and for later entries the revision named in each link (first-order projections, class and counting developments, Tree Evaluation, the October 2026 coverage batches, and entries whose declarations were later renamed or changed). Selection and wording remain open to Samuel's editorial direction.

Unless specified otherwise, encoded decision problems use an injective encoding into finite bit strings, and input size is encoded length. Class statements and reductions refer to the library's machine definitions. Circuit-family results are nonuniform unless uniformity is explicitly asserted. A circuit-size bound does not by itself certify a constructor's runtime; a decoder's correctness does not by itself certify its running time. Assumptions such as one-way functions or circuit hardness are premises of implications, not established existence results.

Source citations credit the mathematics; linked modules record formalization authors and port provenance. This index was prepared and independently reviewed with AI assistance. Lean checks the formal declarations; the prose-to-Lean comparison is a separate review, not a machine-checked equivalence proof.

Contents

1 Completeness and approximation **2**
2 Logic, diagonalization and the polynomial hierarchy **6**
3 Space, alternation and interaction **12**
4 Randomness and counting **17**
5 Cryptographic constructions **31**
6 Circuit depth, lower bounds and barriers **33**
7 Proof complexity **40**
8 Communication and formulas **41**
9 Boolean analysis and expander walks **42**
10 Sensitivity and query complexity **45**
11 Finite combinatorial inequalities **46**
12 Reed–Solomon decoding **47**
13 Arithmetic circuits and simultaneous computation **47**
14 Circuits for iterated functions **48**

1 Completeness and approximation

1.1 Cook–Levin and its complement

SAT is NP-complete and UNSAT is coNP-complete under tidy polynomial-time many-one reductions, using the library's encoding `encCNF` of CNF formulas. Reduction efficiency is proved, not assumed. SAT is also NP-complete when every clause has exactly three literal occurrences: repeated literals are permitted. This does not establish the variant requiring three distinct literals per clause.[7, Section 2.3]

`Cslib.Complexity.CookLevin.SAT_npComplete`
`Cslib.Complexity.CookLevin.UNSAT_coNPComplete`
`Cslib.Complexity.ExactThreeSAT_npComplete`

1.2 Parsimonious Cook–Levin and complete counting problems

The Cook–Levin tableau is parsimonious: for a deterministic multi-tape machine M, input prefix u, certificate length L and step bound T, the tableau has exactly as many satisfying assignments (of the variables occurring in it) as there are words $w \in \{0,1\}^L$ such that M on uw has halted with output 1 after T steps. Hence the existing Cook–Levin reduction of a verifier V maps an instance a to a CNF formula with exactly $|\{w \in \{0,1\}^{p(|a|)} : V(a,w) = 1\}|$ satisfying assignments, and #SAT is #P-complete under parsimonious polynomial-time reductions $f(a) = \#\mathrm{SAT}(r(a))$, for the verifier-based #P (certificates of exact polynomial length) and every injective instance encoding. The count ranges over assignments of the occurring variables. Consequently, under polynomial-time many-one reductions: formulas with an odd number of satisfying assignments form a $\oplus$P-complete problem; unique satisfiability is US-complete; a count not divisible by k is Mod_kP-complete and its complement coMod_kP-complete, for every modulus k (including the degenerate moduli 0 and 1 of the total definition); pairs (φ, m), m in binary, with $\#\varphi \geq m$, as well as MAJSAT (CNF formulas φ satisfied by strictly more than half of the $2^{|\mathrm{vars}(\varphi)|}$ assignments to their occurring variables, $2^{|\mathrm{vars}(\varphi)|} < 2 \cdot \#\varphi$, via an odd/even variable-relabelling and selector-clause padding construction), form PP-complete problems; and pairs with $\#\varphi = m$ form a $\mathsf{C}_=$P-complete problem whose complement is $\mathsf{coC}_=$P-complete. All classes are the library's verifier-based definitions, and variable counts range over the variables occurring in φ. [129][98, Chapter 18][99][18][43][116][15, 24, 132][39]

`Cslib.Complexity.CookLevin.modelCount_tableau`
`Cslib.Complexity.CookLevin.modelCount_reduce`
`Cslib.Complexity.SharpSAT.computableTidilyInPolyTime_card_vars`
`Cslib.Complexity.SharpSAT.mem_SharpP`
`Cslib.Complexity.CookLevin.SharpSAT_sharpPComplete`
`Cslib.Complexity.ParitySAT_parityPComplete`
`Cslib.Complexity.USAT_usComplete`
`Cslib.Complexity.ModSAT_modPComplete`
`Cslib.Complexity.compl_ModSAT_coModPComplete`
`Cslib.Complexity.SharpSATAtLeast_ppComplete`
`Cslib.Complexity.MAJSAT_mem_PP`
`Cslib.Complexity.padMajority_mem_MAJSAT_iff`
`Cslib.Complexity.MAJSAT_ppComplete`
`Cslib.Complexity.SharpSATExact_ceqPComplete`
`Cslib.Complexity.compl_SharpSATExact_coCeqPComplete`

1.3 DP and SAT–UNSAT completeness

The difference class DP consists of languages $B \cap C$ with $B \in \mathsf{NP}$ and $C \in \mathsf{coNP}$, equivalently differences of two NP languages. For any NP-complete B and coNP-complete C, the product language $B \times C$ is DP-complete under polynomial-time many-one reductions, using the product encoding. In particular, pairs of CNF formulas (φ, ψ) with φ satisfiable and ψ unsatisfiable form a DP-complete problem. The theorem quantifies over all injectively encoded source types; the two component reductions and their pairing have certified polynomial time. The definition permits distinct NP and coNP components; it does not require the resulting language to belong to both classes.[98]

`Cslib.Complexity.NPComplete.prod_coNPComplete`
`Cslib.Complexity.SATUNSAT_dpComplete`

The complement class coDP consists of unions of an NP language and a coNP language; it is closed under union and polynomial-time many-one reductions.

`Cslib.Complexity.mem_coDP_iff_union`
`Cslib.Complexity.union_mem_coDP`
`Cslib.Complexity.PolyTimeReducible.mem_coDP`

1.4 The Boolean hierarchy over NP

Using the difference-hierarchy convention,

$$\mathsf{BH}_0 = \{\varnothing\}, \qquad \mathsf{BH}_{n+1} = \{B \setminus D : B \in \mathsf{NP},\ D \in \mathsf{BH}_n\}.$$

We prove $\mathsf{BH}_1 = \mathsf{NP}$ and $\mathsf{BH}_2 = \mathsf{DP}$, monotonicity of levels, complement membership one level higher, and polynomial-time many-one reduction closure at each fixed level. Consequently the union of the finite levels is closed under complement. The construction and inverse-image theorem are generic over the base class of sets; NP supplies the complexity instance. This follows the difference presentation of the Boolean hierarchy; no separation between levels or hierarchy-collapse theorem is asserted.[25]

`Cslib.Complexity.BH_one`
`Cslib.Complexity.BH_two`
`Cslib.Complexity.BH_mono`
`Cslib.Complexity.compl_mem_BH_succ`
`Cslib.Complexity.compl_mem_BooleanHierarchy`
`Cslib.Complexity.PolyTimeReducible.mem_BH`

1.5 Graph and number problems are NP-complete

Three-colorability, undirected Hamiltonian cycle, binary subset sum, and the budget decision version of traveling salesman are NP-complete under certified polynomial-time many-one reductions. Subset sum selects entries of a list of natural numbers, allowing repeated values; its binary encoding does not give strong NP-hardness. TSP uses binary natural distances and budget, and does not require symmetry or the triangle inequality.[42, 71, 118]

`Cslib.Complexity.ThreeColoring_npComplete`
`Cslib.Complexity.HamCycle_npComplete`
`Cslib.Complexity.SubsetSum_npComplete`
`Cslib.Complexity.TSP_npComplete`

1.6 Exponential lower bounds for graph problems under ETH

Assume the exponential time hypothesis. Then for each of the following problems there is $\delta > 0$ such that it is not decidable within time $2^{\delta\mu}$ times a polynomial in the length of the encoded instance: three-colorability, independent set and vertex cover (graph with binary target), and directed and undirected Hamiltonian cycle and path (graph with endpoints for paths), all with $\mu = |V| + |E|$; independent set and clique with $\mu = |V|$. A graph is a list of declared vertices and a list of edges, and $|V|$, $|E|$ are the list lengths with repetitions; they bound the distinct counts, so the bounds for distinct counts follow. The predicate negated allows the decider to depend on δ. Each bound composes a polynomial-time reduction from 3-CNF formulas whose output measure is linear in the number m of clauses with the sparsification lemma, under which ETH excludes $2^{o(m)}$ for 3SAT. Three-colorability uses the gadget reduction (at most $(24k + 12)m + 9$ vertices and edges for clauses of width at most k); independent set, vertex cover and clique use the variable-edge reduction of Garey and Johnson (at most $(k^2+5k)m$), not Karp's occurrence graph, which can have quadratically many edges. Hamiltonian problems use a variant of Sipser's diamond construction with one diamond per distinct variable and one slot per occurrence; its directed path graph has at most $(23k+1)m+1$ vertices and arcs in total, and the cycle and undirected graphs obtained by closing and splitting it stay linear (at most $(115k + 5)m + 10$ for undirected Hamiltonian cycle). The correctness of diamond graphs is proved once for arbitrary row layouts, and Sipser's construction, with $\Theta(m^2)$ vertices in the worst case, is an instance of it. Garey and Johnson's reduction from vertex cover to Hamiltonian cycle is not used: its selector edges are $\Theta(m^2)$ on these instances. [32, 42, 63, 86, 118]

`Cslib.Complexity.ETH.not_decidableInSubexpTime_ThreeColoring`
`Cslib.Complexity.ETH.not_decidableInSubexpTime_IndependentSet`
`Cslib.Complexity.ETH.not_decidableInSubexpTime_IndependentSet_vertices`
`Cslib.Complexity.ETH.not_decidableInSubexpTime_VertexCover`
`Cslib.Complexity.ETH.not_decidableInSubexpTime_Clique`
`Cslib.Complexity.ETH.not_decidableInSubexpTime_HamCycle`
`Cslib.Complexity.ETH.not_decidableInSubexpTime_HamPath`
`Cslib.Complexity.ETH.not_decidableInSubexpTime_DirHamCycle`
`Cslib.Complexity.ETH.not_decidableInSubexpTime_DirHamPath`

The transfer lemma and the diamond theorem hold for arbitrary measures and row layouts.

`Cslib.Complexity.ETH.not_decidableInSubexpTime_of_reduction`
`Cslib.Complexity.Diamond.mem_DirHamPath_iff`
`Cslib.Complexity.SAT.length_sparseDiamondGraph_le`
`Cslib.Complexity.SAT.length_toSparseHamCycle_le`
`Cslib.Complexity.SAT.length_toSparseIndependentSet_le`

Circuit evaluation is P-complete. Given a one-output De Morgan circuit with n inputs and an input in $\{0,1\}^n$, encoded by `encCircuitValue`, deciding whether the output is true is in P. Every problem in P, under any encoding of its instances, reduces to it by a logarithmic-space many-one reduction, so it is P-complete under logspace reductions, and it lies in L exactly when $\mathsf{P} = \mathsf{L}$ for every encoding. Correctness is required on encoded instances. The machine model has read-only input, write-only output and deterministic multi-tape computation over $\{0,1\}$. Logspace charges work-tape cells only, with bound $c(\lfloor \log_2 \max\{m,1\} \rfloor + 1)$ for encoded input length m. The reduction maps an instance a with $m = |\mathrm{enc}(a)|$ to the tableau circuit of a polynomial-time decider that halts within $C(m+1)^d$ steps, run for $T = C(m+1)^d + m + 1$ steps, with $\mathrm{enc}(a)$ as its input bits rather than hardwired constants. The circuit outputs whether the machine has written exactly [true] after T steps. The construction is uniform: each gate is computed from its number within work space linear in the binary lengths of m, T and the number, and a read machine writes the gates in turn. It works directly on multi-tape configurations, with no conversion to single-tape or oblivious machines. Monotone Circuit Value is addressed in the next entry. The NC consequences and first-order-reduction formulation remain outstanding. [7, 79, 98]

`Cslib.Complexity.CircuitValue.logSpaceComplete`
`Cslib.Complexity.CircuitValue.logSpaceHard`
`Cslib.Complexity.CircuitValue.logSpaceReducible_of_mem_P`
`Cslib.Complexity.CircuitValue.mem_L_iff_forall_P_eq_L`
`Cslib.Complexity.CircuitValue.computableInLogSpace_tableauInstance`
`Cslib.Circuits.Boolean.Tableau.eval_circuit`
`Cslib.Circuits.Boolean.Tableau.gateCode_inSpace`
`Cslib.Complexity.CircuitValue.mem_P`

1.7 Monotone Circuit Value

Monotone Circuit Value is P-complete under logarithmic-space many-one reductions. Instances are well-formed circuits over constants and binary AND/OR, with one output and an explicit input assignment, encoded by `encMonotoneCircuitValue`. Every problem in P, for every injective instance encoding, reduces to this problem. Consequently Monotone Circuit Value belongs to L exactly when $\mathsf{P} = \mathsf{L}$ for every instance encoding.

The reduction writes the dual-rail translation of a polynomial-time decider's tableau. Each original gate becomes two monotone gates; the inputs are a true bit followed by each original bit and its negation. The resource proof uses a shared streaming writer. General Circuit Value also reduces to Monotone Circuit Value, through its decider's tableau. No planarity restriction is established. [49]

`Cslib.Complexity.MonotoneCircuitValue.logSpaceComplete`
`Cslib.Complexity.MonotoneCircuitValue.computableInLogSpace_dualTableauInstance`
`Cslib.Complexity.MonotoneCircuitValue.circuitValue_logSpaceReducible`
`Cslib.Complexity.MonotoneCircuitValue.mem_L_iff_forall_P_eq_L`
`Cslib.Circuits.Monotone.eval_dualRail`
`Cslib.Circuits.Monotone.size_dualRail`

1.8 The PCP theorem

$$\mathsf{NP} = \mathsf{PCP}(O(\log n), O(1)).$$

The verifier is polynomial-time and nonadaptive, has perfect completeness and soundness at most $1/2$, and uses logarithmically many random bits and constantly many queries. In the Lean equality the bound functions are $\lfloor \log_2 n \rfloor + 1$ (with Lean's natural-number logarithm) and 1; the class definition allows constant multiples of both. It does not assert one query.[33]

`Cslib.Complexity.NP_eq_PCP_log`

1.9 A constant gap for MAX-3SAT

It is NP-hard, under polynomial-time promise reductions, to distinguish a satisfiable formula with clauses of at most three literals from one in which every assignment satisfies at most the fraction

$$1 - \frac{1}{3 \cdot 2^{77}}$$

of its clauses. Clauses are counted with multiplicity. This is the explicit gap obtained from the formalized PCP construction; no optimal approximation threshold is asserted.[8, 33]

`Cslib.Complexity.GapThreeSAT_npHard`

1.10 Constant gaps for Independent Set, Clique and Vertex Cover

For graphs given by declared vertices and edges and a target k, it is NP-hard, under polynomial-time promise reductions, to distinguish $\alpha(G) \geq k$ from $\alpha(G) \leq (1 - \frac{1}{3 \cdot 2^{77}})k$, likewise for the clique number, and to distinguish $\tau(G) \leq k$ from $\tau(G) \geq (1 + \frac{1}{3 \cdot 2^{78}})k$ for the vertex cover number. Values are taken over declared vertices. The reductions apply Karp's occurrence graph to the MAX-3SAT gap (an independent set yields an assignment satisfying as many clauses); for vertex cover they pass through MAX-E3SAT with repeated literals allowed, whose graphs have exactly $3m$ vertices. This is not the FGLSS construction; the constants are explicit and far from optimal, and amplification to every constant is not proved.[8][7, Chapter 11]

`Cslib.Complexity.GapExactThreeSAT_npHard`
`Cslib.Complexity.GapIndependentSet_npHard`
`Cslib.Complexity.GapClique_npHard`
`Cslib.Complexity.GapVertexCover_npHard`

1.11 PCP without random bits

For the same nonadaptive polynomial-time verifier model,

$$\mathsf{PCP}(0,0) = \mathsf{P}, \qquad A \in \mathsf{NP} \iff \exists k \in \mathbb{N},\ A \in \mathsf{PCP}\big(0, n \mapsto (n+1)^k\big).$$

The class definition allows constant multiples of the query bound. With no random bits, perfect completeness and soundness at most $1/2$ force deterministic acceptance of an appropriate proof for a member and rejection of every proof for a nonmember. These identities hold for each injective bit-string encoding. [7, Chapter 11]

`Cslib.Complexity.PCP_zero_zero`
`Cslib.Complexity.mem_NP_iff_exists_PCP_zero`

2 Logic, diagonalization and the polynomial hierarchy

2.1 Fagin's theorem

For a finite relational vocabulary, a class of finite structures is definable by an existential second-order sentence exactly when it is closed under isomorphism and its encoded query language belongs to NP. The encoding gives universe size in unary and framed relation truth tables; malformed tables are rejected. Both directions connect the logical semantics to the machine-backed verifier class; structures of cardinality zero and one are included.[35, 60]

`Cslib.Logic.ESO.definable_iff_isoClosed_and_queryLanguage_mem_NP`

2.2 SAT completeness via order-free first-order interpretations

All reductions in this entry are first-order interpretations of finite relational structures of finite relational vocabularies without built-in order or numeric predicates. CNF formulas have binary relations $\mathrm{pos}(c, x)$ and $\mathrm{neg}(c, x)$; each element names a clause and a variable, and a clause related to nothing is empty. SAT, the satisfiable structures, has its encoded query language in NP. Every isomorphism-closed class whose query language is in NP and which contains every empty structure reduces to SAT by grounding an existential second-order definition in Tseitin style. Empty structures may differ on their nullary relations. For classes containing every empty structure, reducibility to SAT is equivalent to isomorphism closure and NP membership; every class reducible to SAT has its query language in NP. Without the empty-structure hypothesis, correctness is proved on nonempty input structures. [60, Chapter 7][35]

3SAT (at most three literals per clause) and 3COL (loops allowed) are interreducible, and 3SAT reduces to the class of simple three-colorable graph structures; simplicity is guaranteed on yes-instances. SAT reduces to independent set on structures with at least two elements, using Karp's occurrence graph. Independent set, clique and vertex cover, with target size given by the cardinality of a unary relation, are existential second-order definable. [42, 71]

These are order-free variants, not completeness assertions under the standard ordered projections. Positive-dimensional interpretations send the empty structure to the empty, satisfiable CNF structure. The SAT-to-independent-set reduction cannot be extended to singleton inputs with these encodings: every singleton target is a yes-instance, whereas an empty clause on one element is unsatisfiable. SAT-to-3SAT, hardness of 3SAT, 3COL, independent set, clique and vertex cover in this interpretation framework, and bridges to the encoded problems remain outstanding.

`Cslib.Logic.ESO.Definable.reduces_SAT`
`Cslib.Logic.ESO.definable_iff_reduces_SAT`
`Cslib.Logic.ESO.Definable.exists_interpretation_SAT_of_nonempty`
`Cslib.Logic.FO.cnf.queryLanguage_SAT_mem_NP`
`Cslib.Logic.FO.cnf.reduces_SAT_of_queryLanguage_mem_NP`
`Cslib.Logic.FO.cnf.exists_interpretation_SAT_of_queryLanguage_mem_NP`
`Cslib.Logic.FO.cnf.reduces_SAT_iff_isoClosed_and_queryLanguage_mem_NP`
`Cslib.Logic.FO.cnf.reduces_ThreeSAT_ThreeColorable`
`Cslib.Logic.FO.cnf.reduces_ThreeSAT_SimpleThreeColorable`
`Cslib.Logic.FO.graph.reduces_ThreeColorable_ThreeSAT`
`Cslib.Logic.FO.cnf.reducesNontrivial_SAT_IndependentSet`
`Cslib.Logic.FO.cnf.queryLanguage_mem_NP_of_reduces_SAT`
`Cslib.Logic.FO.cnf.not_reduces_SAT_IndependentSet`
`Cslib.Logic.FO.sizedGraph.definable_IndependentSet`
`Cslib.Logic.FO.sizedGraph.definable_Clique`
`Cslib.Logic.FO.sizedGraph.definable_VertexCover`

2.3 Canonical numeric structures and constant-aware interpretations

For every $n \geq 2$, an input structure on $[n] = \{0, \ldots, n-1\}$ has a canonical expansion by $\leq$, BIT and the constants 0, 1 and $\max = n - 1$. The constants 0 and 1 are distinct. On arbitrary terms s, i, BIT has the exact semantics

$$\mathrm{BIT}(s, i) \quad \Longleftrightarrow \quad \text{bit } \mathrm{val}(i) \text{ of } \mathrm{val}(s) \text{ is } 1,$$

where bit zero is least significant. Numeric interpretations are computed from the universe size, rather than supplied as input tables. Input functions, constants and relations retain their interpretations, and embedding any input formula, including quantified formulas, preserves truth under every assignment. The construction uses the existing first-order terms and formulas.

For relational input vocabularies, forgetting the numeric expansion is a one-dimensional interpretation whose output is isomorphic to the input; its sentence pullback preserves truth.

More generally, let I be a relational interpretation of finite dimension and let finitely many target constants be specified by source formulas. If each formula selects exactly one tuple inside the interpreted domain, the resulting structure with constants admits formula and sentence pullback preserving truth under every free-variable assignment. This existence/uniqueness condition is an explicit hypothesis; definitions with no value or multiple values are not assigned defaults. Source languages may already contain functions and constants; the target language is relational with the specified additional constants. For every canonical expansion with $n \geq 2$, equality to the source terms defines target 0, 1 and max with values 0, 1 and $n - 1$.

Canonical BIT lifting on base-n tuples, numeric-preserving composition and the ordered logspace/capture results remain open. [60, Section 1.2, Provisos 1.14–1.15, Definition 1.26 and Proposition 3.5] The earlier canonical-expansion interface is credited separately. [111]

`Cslib.Logic.FO.Ordered.realize_bit`
`Cslib.Logic.FO.Ordered.realize_inputFormula`
`Cslib.Logic.FO.Ordered.realize_forget_onSentence`
`Cslib.Logic.FO.Interpretation.WithConstants.realize_onFormula`
`Cslib.Logic.FO.Interpretation.WithConstants.realize_onSentence`
`Cslib.Logic.FO.Ordered.constants_valid`
`Cslib.Logic.FO.Ordered.constants_value_val`

2.4 Lexicographic tuple order, successor and canonical constants

For every base $n \geq 2$ and dimension $k \geq 0$, the full tuple universe $[n]^k$ is in bijection with $[n^k]$ through the rank

$$\operatorname{rank}(x) = \sum_{i=0}^{k-1} x_i n^{k-1-i}.$$

The inverse recovers coordinate i by division by n^{k-1-i} and remainder modulo n. Numerical rank order is exactly the first-coordinate-most-significant lexicographic order: $\operatorname{rank}(x) < \operatorname{rank}(y)$ iff some coordinate i satisfies $x_i < y_i$ and every earlier coordinate agrees. The construction reuses the existing tuple enumeration by reversing coordinates; it works for arbitrary bases, including nonpowers of two.

Native quantifier-free formulas over the canonical source structure define both strict and weak tuple order with these exact rank semantics. For positive dimension, coordinate equalities define the target constants

$$(0, \ldots, 0), \qquad (0, \ldots, 0, 1), \qquad (n-1, \ldots, n-1),$$

with ranks 0, 1 and $n^k - 1$. Their existence and uniqueness are proved in the full tuple interpretation, so the constant-aware formula/sentence pullback applies. At dimension zero, the rank bijection still exists and its universe is a singleton; no two distinct constants are asserted there.

A native successor formula holds exactly when $\operatorname{rank}(y) = \operatorname{rank}(x) + 1$. Its carry characterization increments one coordinate, keeps earlier coordinates equal, and changes every later digit from $n-1$ to 0. It rejects all pairs in dimension zero and rejects every successor of the maximum tuple, including wraparound to zero. The generic carry compiler is quantifier-free whenever its supplied source successor formula is quantifier-free, as in Exercise 1.33(2) with source SUC available. In the library's $\leq$/BIT basis, the canonical source successor instead uses one universal element quantifier to express adjacency; the compiled tuple formula has one such quantifier in each carry alternative. No unconditional quantifier-free claim is made for this basis.

This completes the full-domain order, constant and successor prerequisites of Immerman's Remark 1.32 and Exercise 1.33(1–2), with the stated source-basis convention. Canonical BIT lifting, numeric-preserving composition and the ordered complexity/capture endpoints remain open. No automatic BIT definition for a restricted tuple domain is claimed. [60, Remark 1.32 and Exercise 1.33]

`Cslib.Logic.FO.Ordered.lexEquiv`
`Cslib.Logic.FO.Ordered.lexRank_lt_iff_lex`
`Cslib.Logic.FO.Ordered.realize_tupleLe`
`Cslib.Logic.FO.Ordered.isQF_tupleLe`
`Cslib.Logic.FO.Ordered.lexRank_tupleConstants_value`
`Cslib.Logic.FO.Ordered.realize_tupleSucc`
`Cslib.Logic.FO.Ordered.isQF_tupleSuccOf`

2.5 First-order model checking in logarithmic space

For a finite relational vocabulary and a first-order sentence φ whose atomic formulas are relation symbols and equality applied to variables, the encoded query language of the finite models of φ is in L; hence the query language of every first-order definable class of finite structures is in L. The encoding gives the universe size in unary, then one framed truth table per relation symbol in the order of the vocabulary, entries indexed by the tuple index; malformed inputs are rejected, and the empty structure and nullary relations are included. The decider is a counter program with random access to its input: it checks the shape of the input, then evaluates φ with one register per bound variable, reading table bits by index arithmetic. Counter programs whose registers stay polynomially bounded, and more generally machines querying input bits with logarithmic states, run in logarithmic space by a multi-pass simulation.[60, Theorem 3.1]

Immerman's FO also has constant symbols and the built-in predicates $\leq$ and BIT; these are not covered (a relational language has no constants). The next entry proves the order-free interpretation-to-logspace bridge.

`Cslib.Logic.Vocabulary.queryLanguage_mem_L`
`Cslib.Logic.FO.Definable.queryLanguage_mem_L`
`Turing.QueryMachine.computableInLogSpace`
`Turing.CounterProgram.computableInLogSpace`
`Turing.CounterProgram.computableInLogSpace_of_reachesOut`

2.6 Order-free interpretations induce logspace reductions

For finite relational vocabularies, every fixed finite-dimensional first-order interpretation without built-in order induces a total logarithmic-space transducer on encoded structures. The encoding gives universe size in unary and framed relation truth tables. The transducer counts domain tuples and streams the interpreted relation tables, reconstructing elements by their ranks instead of storing the output.

Consequently, an order-free interpretation reduction from K to an isomorphism-closed class K' induces a logspace many-one reduction between their encoded query languages. Membership in L or P therefore transfers from the target query language to the source. Parsed structure data with invalid table sizes map to fixed invalid target data; this is the shape check within the encoding-relative contract. Empty universes, empty interpreted domains, nullary relations, and dimension-zero interpretations are included. These statements concern the order-free general layer; the standard ordered FO($\leq$, BIT) results are not covered here. [60, 118]

`Cslib.Logic.FO.Interpretation.computableInLogSpace_encodedMap`
`Cslib.Logic.FO.Reduces.logSpaceReducible`
`Cslib.Logic.FO.Reduces.queryLanguage_mem_L`
`Cslib.Logic.FO.Reduces.queryLanguage_mem_P`

First-order projection infrastructure. With separate numeric and input vocabularies, each projection formula selects a constant or one signed input-relation bit using numeric tests alone. For a fixed numeric structure and variable assignment, the same selected literal works for every interpretation of the input relations. The finite numeric cases are mutually exclusive and exhaustive. Projections with finite coordinate types compose, with the composite structure isomorphic to successive interpretation; quantifier-free numeric definitions and guards remain quantifier-free under composition. A concrete client is loop-free graph complement. [61, Definitions 3.3–3.6]
This is a vocabulary-parametric, relativized framework: output domains may be restricted by numeric formulas, and interpreted vocabularies are relational. It does not yet specialize to positive-dimensional full tuple universes with canonical order and BIT, certify a logspace transducer, or prove Circuit Value completeness under first-order projections. The classical quantifier-free convention of the cited source excludes BIT.
`Cslib.Logic.FO.ProjectionFormula.exists_literal`
`Cslib.Logic.FO.Projection.compEquiv`
`Cslib.Logic.FO.Projection.IsQF.comp`

Quantifier-free reductions and their automation. A class of finite structures is quantifier-free reducible to another when a first-order interpretation whose domain and relation formulas are quantifier-free maps the members of the first class into the second and the other finite structures outside it. Quantifier-free reducibility implies first-order reducibility and is reflexive on isomorphism-closed classes of a relational vocabulary and transitive; for a one-dimensional interpretation whose domain formula always holds, correctness may be stated on the input universe itself. The tactics `fo_qf`, `fo_iso_closed`, `fo_simp` and `fo_reduction` prove quantifier-freeness and isomorphism closure and reduce a reduction goal to its semantic statement. As clients, independent set, clique and vertex cover, posed as a graph with a unary relation whose cardinality is the target size, are pairwise quantifier-free reducible; Karp's clique-to-node-cover reduction is the composite of complementing the edges off the diagonal and complementing the size relation. Edge relations need not be symmetric and loops are ignored; on simple graphs these are the usual problems.[42, 60, 71]
These reductions use no order or other numeric predicates, and they are a different notion from the quantifier-free projections of the previous entry, which select a single input literal for each output relation. The structure classes are not connected here to the encoded NP-complete problems.
`Cslib.Logic.FO.QFReduces.reduces`
`Cslib.Logic.FO.QFReduces.trans`
`Cslib.Logic.FO.QFReduces.of_unitStructure`
`Cslib.Logic.FO.sizedGraph.qfReduces_IndependentSet_Clique`
`Cslib.Logic.FO.sizedGraph.qfReduces_IndependentSet_VertexCover`
`Cslib.Logic.FO.sizedGraph.qfReduces_Clique_VertexCover`

2.7 Polynomial time is strictly smaller than exponential time

For languages of finite bit strings,

$$\mathsf{P} \subsetneq \mathsf{EXP}.$$

The separating language is given by a machine diagonalization. Here EXP is the union of deterministic time classes with bounds 2^{n^k}. The supporting time hierarchy uses a polynomial simulation gap; this entry does not claim the sharp $f \log f$ hierarchy.[7, Chapter 3]
`Cslib.Complexity.P_ssubset_EXP`

2.8 Ladner's theorem

If $\mathsf{P} \neq \mathsf{NP}$, there exists a language of bit strings in NP which is neither in P nor NP-complete under polynomial-time many-one reductions. The witness is a padded satisfiability language with a polynomial-time padding exponent. This is a conditional intermediate-degree theorem; it does not assume an intermediate problem as additional input.[7, Theorem 3.3]
`Cslib.Complexity.Ladner.exists_NP_not_P_not_NPComplete`

2.9 A Baker–Gill–Solovay oracle separation

There exists an oracle language A such that

$$\mathsf{P}^A \subsetneq \mathsf{NP}^A.$$

The classes use the library's certified polynomial-time oracle programs and oracle verifiers. This is the separating-oracle half of the relativization phenomenon, rather than a claim here of both a separating and a collapsing oracle.[11]

`Cslib.Complexity.exists_PWithOracle_ssubset_NPWithOracle`

2.10 Karp–Lipton collapse

If SAT has polynomial-size Boolean circuits on its encoded inputs, then

$$\mathsf{PH} = \Sigma_2^p.$$

In particular this follows if every NP problem, under its encoding, is the preimage of a language in P/poly. The polynomial-time computation of the self-reduction and evaluation of circuit advice are part of the development. The circuit hypothesis is nonuniform.[72]

`Cslib.Complexity.PH_eq_SigmaP_two_of_SAT`
`Cslib.Complexity.PH_eq_SigmaP_two_of_NP_subset_PPoly`

2.11 SAT in BPP collapses the hierarchy

If SAT $\in \mathsf{BPP}$ (in particular if $\mathsf{NP} \subseteq \mathsf{BPP}$ for every encoding), then $\mathsf{PH} = \Sigma_2^p$ for every encoding: Adleman's theorem gives SAT polynomial-size circuits and the Karp–Lipton collapse applies.[2, 72]

`Cslib.Complexity.PH_eq_SigmaP_two_of_SAT_mem_BPP`
`Cslib.Complexity.PH_eq_SigmaP_two_of_NP_subset_BPP`

2.12 Plain Kolmogorov complexity is uncomputable

Let $K(x)$ be the minimum bit-string description length of $x \in \mathbb{N}$ under the library's universal partial-computable description method. Then $K(x) \to \infty$ as $x \to \infty$, but every total computable unbounded function $f : \mathbb{N} \to \mathbb{N}$ satisfies $K(x) < f(x)$ for infinitely many x. In particular, K is not computable. This is plain complexity, often denoted C in the literature, not prefix-free complexity.[83]

`Cslib.Kolmogorov.DescriptionMethod.tendsto_complexity_universal`
`Cslib.Kolmogorov.DescriptionMethod.frequently_complexity_universal_lt_of_computable`
`Cslib.Kolmogorov.DescriptionMethod.not_computable_complexity_universal`

3 Space, alternation and interaction

Containment map. The proved machine-class relationships include

$$\mathsf{NL} \subseteq \mathsf{P} \subseteq \mathsf{NP} \subseteq \mathsf{PH} \subseteq \mathsf{PSPACE} \subseteq \mathsf{EXP}, \qquad \mathsf{AP} = \mathsf{NPSPACE} = \mathsf{PSPACE}.$$

The equalities and the NL inclusion receive entries below. The links here supply the remaining arrows, including PH containment in polynomial space and the deterministic exponential-time simulation of polynomial space. Inclusion arrows do not assert strictness; the separate $\mathsf{P} \subsetneq \mathsf{EXP}$ theorem appears in the preceding section. [7, 118]

`Cslib.Complexity.P_subset_NP`
`Cslib.Complexity.NP_subset_PH`
`Cslib.Complexity.PH_subset_PSPACE`
`Cslib.Complexity.PSPACE_subset_EXP`

3.1 Nondeterministic logspace lies in polynomial time

$$\mathsf{NL} \subseteq \mathsf{P}.$$

NL is defined by native nondeterministic machines with logarithmic space bounded on every computation prefix, every branch halting, and acceptance by an accepting path, as in Sipser's Definition 8.1. Formally, on each input some number of steps bounds every path; a path may end stuck, and then rejects. The polynomial-time decider does not use halting: every problem recognized in logarithmic space by such a machine whose branches may run forever is also in P. This entry does not assert the still-outstanding native $\mathsf{NL} = \mathsf{coNL}$ theorem.[118, Section 8.5]

`Cslib.Complexity.NL_subset_P`
`Cslib.Complexity.mem_P_of_recognizesInSpace`

3.2 Undirected connectivity by short walks in Reingold's graph

For every simple graph G on $\{0, \dots, n-1\}$ and all vertices s, t, t is reachable from s if and only if some walk of length $T(n) = 4(2\lceil \log_2 n \rceil + 65\,\ell(n))$ in the 257^8-regular graph $G_{\ell(n)}$, starting at a fixed vertex over the slot (s, s), ends over a slot of t, where $\ell(n) = 16\lceil \log_2 n \rceil + 524$. Here G_0 is the slot graph of G (each vertex replaced by a cycle of n slots, slot (v, w) joined to (w, v) when $v \sim w$), padded with self-loops to 257^8 labels, and $G_{i+1} = ((G_i \mathbin{ⓩ} H)^2)$ with labels renamed, for the arithmetic zig-zag base H over $\mathbb{Z}/257$ with $\lambda(H) \le 1/5$. The generic statement assumes $\lambda(H) \le 1/5$, at least half self-loops at each vertex, $2|V|^2|B|^2 5^\ell \le 5 \cdot 6^\ell$ and $|V||B|^\ell 4^T \le 5^T$. Each step turns a spectral gap γ into at least $\min(6\gamma/5, 1/5)$ (Reingold takes the eighth power of $G \mathbin{ⓩ} H$ and obtains $\lambda \mapsto \max(\lambda^2, 1/2)$; the constant differs, the $O(\log n)$ depth does not), a connected component with half self-loops has gap at least $1/(2N^2|B|^2)$, and walks of length t connect every pair once $\lambda^t(N-1) < 1$. This is the combinatorial part of the proof of USTCON $\in$ L; the class-level theorem is the next entry.[106, Definition 3.1, Lemmas 3.2–3.3, Theorem 4.1][7, Section 21.4]

`Cslib.Reachability.reachable_iff_exists_reingoldGraph`
`Cslib.RotationMap.reachable_iff_exists_root_neighbor_pow_reingold`
`Cslib.RotationMap.min_pow_le_one_sub_spectralExpansion_reingold`
`Cslib.RotationMap.spectralExpansion_le_of_preconnected`
`Cslib.RotationMap.exists_neighbor_pow_eq`

3.3 Undirected connectivity is in logarithmic space, and $\mathsf{SL} = \mathsf{L}$

$$\mathrm{USTCON} \in \mathsf{L}, \qquad \mathsf{SL} = \mathsf{L}.$$

Instances are simple graphs on $\{0, \ldots, n-1\}$ given by a symmetric adjacency matrix with zero diagonal and two vertices s, t, encoded by the unary header $1^n 0 1^s 0 1^t 0$ followed by the matrix row by row; the instance is accepted when t is reachable from s. L is the library's class: a deterministic binary multi-tape machine with finitely many states computes the decision within space $c(\lfloor \log_2 m \rfloor + 1)$ on encodings of length m, the input and output not charged; as for every problem in L, correctness is required only on encodings of instances, and recognizing encodings is not part of the statement. The decider walks in Reingold's graph G_ℓ (depth $16a + 524$, walk length $4(2a + 65(16a + 524))$ for a the binary length of n, 257^8-regular) along the base-257^8 digits of every counter below 257^{8T}, evaluating the rotation map in place with one program counter and one label per level and reading the matrix only at a step of the slot graph. The transformation is a variant of Reingold's Definition 3.1 with the square of the zig-zag product instead of the eighth power; the constants are far from optimal. Symmetric logarithmic space SL is defined by its logspace many-one reduction characterization as the class of problems logspace many-one reducible to USTCON; combining $\mathrm{USTCON} \in \mathsf{L}$ with the logspace reduction of any problem in L to USTCON via a canonical two-vertex witness yields logspace completeness of USTCON for SL and L and Reingold's corollary $\mathsf{SL} = \mathsf{L}$. The symmetric Turing machine model and its equivalence with logspace reducibility to USTCON are not separately formalized, and the encoding is the adjacency matrix rather than the edge-list encoding of the library's other graph problems.[82][106, Section 1, the proof of Lemma 3.4, and Theorems 4.1–4.2][7, Section 21.4]

`Cslib.Complexity.USTCON_mem_L`
`Cslib.Complexity.USTCON_logSpaceComplete_SL`
`Cslib.Complexity.USTCON_logSpaceComplete_L`
`Cslib.Complexity.SL_eq_L`
`Cslib.USTCON.computableInLogSpace_decide`
`Cslib.RotationMap.exists_iterate_reingoldEvalStep`
`Cslib.Reachability.Reingold.reachable_iff_exists_walk_of_le_two_pow`

3.4 Savitch's theorem for polynomial space

$$\mathsf{NPSPACE} = \mathsf{PSPACE}.$$

Both classes are machine-defined. Nondeterministic acceptance is by an accepting path, with every path respecting the space bound and every branch halting (on each input some number of steps bounds every path; a stuck path rejects); the deterministic decider is total. Halting is not used: Savitch's decider also places in PSPACE every problem recognized in polynomial space by a machine whose branches may run forever. This is the equality of polynomial-space classes, not a statement of the fully parameterized $\mathsf{NSPACE}(s) \subseteq \mathsf{DSPACE}(s^2)$ theorem. [118, Corollary 8.6]

`Cslib.Complexity.NPSPACE_eq_PSPACE`
`Cslib.Complexity.mem_PSPACE_of_recognizesInSpace`

3.5 TQBF is PSPACE-complete

The true closed quantified Boolean formulas are PSPACE-complete under polynomial-time many-one reductions. Membership uses an explicit-stack evaluator; hardness includes a certified polynomial-time construction of the quantified reachability formula. Formulas with free variables are excluded from TQBF. [118, Theorem 8.9]

`Cslib.Complexity.TQBF_pspaceComplete`

3.6 Generalized Geography is PSPACE-complete

Generalized Geography in its vertex version (a directed graph with declared vertices and a start vertex; players alternately move a token along arcs to unvisited declared vertices; the player unable to move loses; the instance is accepted when the first player wins, by backward induction) is PSPACE-complete under polynomial-time many-one reductions. Membership is a polynomial-time game verifier for the alternating characterization of PSPACE. Hardness reduces true closed formulas $\exists x_1 \forall y_1 \cdots \exists x_n \forall y_n \, \psi$ with ψ in CNF (shown PSPACE-complete by normalizing the formulas of games; the game on them is Sipser's FORMULA-GAME) by the diamond construction of Sipser's proof, which adapts Schaefer's technique. Formulas with free variables are rejected. Schaefer's arc version of the game is not treated.[118, Theorems 8.11, 8.14][84, 107]

`Cslib.Complexity.TQBFCNF_pspaceComplete`
`Cslib.Complexity.GeneralizedGeography_mem_PSPACE`
`Cslib.Complexity.GeneralizedGeography_pspaceHard`
`Cslib.Complexity.GeneralizedGeography_pspaceComplete`

3.7 Alternating polynomial time equals polynomial space

$$\mathsf{AP} = \mathsf{PSPACE}.$$

AP uses the library's native alternating machines and a polynomial time bound. The equality is with the same machine-defined PSPACE used above; it is not merely an equality of abstract game predicates.[27]

`Cslib.Complexity.AP_eq_PSPACE`

3.8 Interactive proofs lie in polynomial space

$$\mathsf{IP} \subseteq \mathsf{PSPACE}.$$

IP uses a private-coin verifier realized by one fair-coin oracle machine and a polynomial clock valid against every stateful prover. Provers may be randomized and computationally unbounded; completeness is at least $2/3$ and soundness at most $1/3$. The converse $\mathsf{PSPACE} \subseteq \mathsf{IP}$ is the entry "Interactive proofs capture polynomial space" below. [7, Chapter 8]

`Cslib.Complexity.IP_subset_PSPACE`

3.9 The algebraic protocol for quantified Boolean formulas

Let φ be a closed quantified Boolean formula with $|\varphi|$ syntax nodes, and let E be its arithmetization over a commutative ring, with Shen's linearization operator applied to every variable of a list xs after every quantifier. Then $E(0) = 1$ if φ is true and $E(0) = 0$ if it is false. If xs contains the variables of φ, every syntactic degree bound of E is at most $|\varphi|$, and the round degree (the sum of these bounds over the operator nodes) is at most $(2|xs| + 1)|\varphi|^2$. A Shamir–Shen-style verifier checks the claim $E(0) = 1$ by recursion on E: at a binary connective the prover sends the two values, and at $\exists x$, $\forall x$ or L_x it sends the univariate restriction, truncated to the degree bound, and the verifier draws a random point. The honest prover is accepted with probability 1 on every true claim. Over an integral domain, if the sampler gives each element probability at most ε, every stateful, adaptive, randomized prover is accepted on a false claim with probability at most (round degree)$\cdot\varepsilon$. For TQBF with uniform samples from a finite field F, false closed formulas are accepted with probability at most $3|\varphi|^3/|F|$. The verifier is an abstract typed oracle program with exact sampling; its realization in the IP machine model is the next entry. [88, 113, 115][7, Chapter 8]

`Cslib.Logic.PL.QBF.eval_arithmetize`
`Cslib.Logic.PL.QBF.eval_arithmetize_zero`
`Cslib.Logic.PL.QBF.deg_arithmetize_le`
`Cslib.Logic.PL.QBF.roundDegree_arithmetize_le`
`Cslib.Complexity.QBFProtocol.acceptProb_honest_verify`
`Cslib.Complexity.QBFProtocol.acceptProb_verify_le`
`Cslib.Complexity.QBFProtocol.acceptProb_honest_of_mem_TQBF`
`Cslib.Complexity.QBFProtocol.acceptProb_le_of_notMem_TQBF`

3.10 Interactive proofs capture polynomial space

$$\mathsf{IP} = \mathsf{PSPACE}, \qquad \mathsf{coNP} \subseteq \mathsf{PH} \subseteq \mathsf{IP},$$

for every encoding of instances. TQBF (closed quantified Boolean formulas over $\mathbb{N}$-indexed variables, not necessarily prenex) has an interactive proof with completeness 1 and soundness error $1/3$ against every stateful randomized prover, in the library's IP model: private fair coins, one oracle machine with a polynomial clock. On a formula with s syntax nodes the verifier rejects formulas with a free variable, arithmetizes with Shen's linearization of every variable occurring in the tokens after every quantifier, chooses the greatest prime $p \le 24s^2(s+1)$ by trial division (above $12s^2(s+1)$ by Bertrand's postulate), and runs a Shamir–Shen-style protocol as a stack machine over the tokens, reading $2s+1$ coefficients per round, with field elements drawn from fair coins with point masses at most $2/p$. Soundness transfers from typed provers to word provers by an exact simulation. With the existing PSPACE-completeness of TQBF and the closure of IP under polynomial-time (Karp) reductions, $\mathsf{PSPACE} \subseteq \mathsf{IP}$; with the existing $\mathsf{IP} \subseteq \mathsf{PSPACE}$, equality follows. The linearized variables are not deduplicated, which costs only polynomially more rounds. [88, 113, 115][7, Section 8.3]

`Cslib.Complexity.IP_eq_PSPACE`
`Cslib.Complexity.PSPACE_subset_IP`
`Cslib.Complexity.coNP_subset_IP`
`Cslib.Complexity.PH_subset_IP`
`Cslib.Complexity.TQBF_mem_IP`
`Cslib.Complexity.QBFProtocol.isInteractiveProof_verifier`
`Cslib.Complexity.QBFProtocol.isOraclePPTOn_verifier`
`Cslib.Complexity.PolyTimeReducible.mem_IP`

3.11 Graph non-isomorphism has a private-coin interactive proof

Graph non-isomorphism belongs to IP. The strict polynomial-time fair-coin verifier has perfect completeness and soundness at most $(17/32)^2 < 1/3$ after two rounds. It samples relabelings by sorting random vertex keys; this avoids assuming exact uniform sampling of a permutation from a fixed number of fair bits. The result is for private coins; the public-coin AM protocol is in the Arthur–Merlin entry. [48]

`Cslib.Complexity.GraphNonIsomorphism.isInteractiveProof_verifier`
`Cslib.Complexity.GraphNonIsomorphism.mem_IP`

3.12 Cook–Mertz Tree Evaluation in small space

A valid instance is a perfect binary tree with every leaf at depth H, with ℓ-bit values at its leaves and a gate $\{0,1\}^\ell \times \{0,1\}^\ell \to \{0,1\}^\ell$ at each internal node. The injective encoding gives ℓ, H in unary, followed by gate truth tables and leaf values in preorder. Write n for its bit length and $b(k)$ for the length of the library's binary encoding of k. One fixed deterministic multi-tape Turing machine computes the root value on every encoded instance, using work space

$$O\big((\ell + H + 1)(b(\ell) + 1) + b(n + 1)\big),$$

that is, $O((\ell + H + 1)\log(\ell + 2) + \log(n + 2))$. The theorem allows zero value length and zero height. Input addressing costs are included, the output is streamed, and a running-time bound exists without an asserted asymptotic rate.

Running once over every nonzero field element, the algebraic procedure preserves the other two registers while adding the scaled tree value to its target register over any finite field with $2\ell < |K| - 1$. The concrete machine chooses a suitable prime, reads the encoded gates, and satisfies a proved state-size invariant. This formalizes the Cook–Mertz construction with Goldreich's value-length parametrization and exposition; the Lean development is by Christian Reitwiessner with disclosed AI assistance.[30, 45] Correctness and resource bounds apply to valid encoded instances; malformed-input rejection is not proved. A separate $O(\log n \log\log n)$ corollary and the refined $O(\ell + H\log(\ell+2))$ parameter bound are not asserted here.

`Cslib.TreeEval.computableTidilyInSpace_eval`
`Cslib.TreeEval.encode_injective`
`Cslib.TreeEval.proc_eq`

4 Randomness and counting

4.1 Sipser–Gács–Lautemann

$$\mathsf{BPP} \subseteq \Sigma_2^p \cap \Pi_2^p.$$

For the library's bounded-error polynomial-time class, membership has both an existential–universal and a universal–existential characterization with polynomially bounded bit-string witnesses and a polynomial-time predicate. [80, 117]

`Cslib.Complexity.BPP_subset_SigmaP_two_inter_PiP_two`

As a consequence of the hierarchy collapse, if $\mathsf{NP} \subseteq \mathsf{P}$ for every type and injective bit-string encoding, then $\mathsf{BPP} = \mathsf{P}$ for every such encoded problem class. The formal premise quantifies over all encodings, rather than assuming equality only at one chosen encoding.[7]

`Cslib.Complexity.BPP_eq_P_of_NP_subset_P`

4.2 Arthur–Merlin classes

For every fixed injective encoding,

$$\mathsf{NP} \cup \mathsf{BPP} \subseteq \mathsf{MA} \subseteq \mathsf{SBP} \subseteq \mathsf{AM} \subseteq \Pi_2^p, \qquad \mathsf{MA} \subseteq \mathsf{AM}, \qquad \mathsf{MA} \subseteq \Sigma_2^p \cap \Pi_2^p,$$

and graph non-isomorphism is in AM. MA and AM are two-message public-coin classes given by deterministic polynomial-time verifiers reading one concatenated string: Merlin's proof of $q(n)$ bits and Arthur's $p(n)$ coins, in that order for MA and in the reverse order for AM, with $p, q \in \mathbb{N}[X]$ and exact message lengths; completeness is at least $2/3$ and soundness at most $1/3$, as counts of random strings. Both classes are closed under polynomial-time many-one reductions; majority votes reduce the error to $2^{-e(n)}$ for every polynomial e (for MA with the proof length fixed first), and AM is closed under polynomially bounded existential quantification. The set lower bound protocol of Goldwasser and Sipser is formalized for sets S of $N(n)$-bit strings with polynomial-time certificates: a threshold $K > 0$ computed in binary in polynomial time with $K \le |S|$ on members and $|S| \le K/2$ on non-members places the problem in AM, via pairwise-independent affine hashing over $\mathbb{F}_2$; with no certificates this gives $\mathsf{SBP} \subseteq \mathsf{AM}$. For graph non-isomorphism (graphs are the simple graphs induced on declared vertices, encoded as vertex and edge lists with binary naturals) the set is the pairs of a relabelling of one adjacency matrix and an automorphism of it, of size $2n!$ or $n!$ by orbit–stabilizer; pairs with different numbers of vertices are accepted outright. The Boppana–Håstad–Zachos theorem: if $\mathsf{coNP} \subseteq \mathsf{AM}$ for every encoding, then for every encoding $\Sigma_2^p = \Pi_2^p = \mathsf{AM}$ and $\mathsf{PH} = \Sigma_2^p$; in particular, if graph isomorphism is NP-complete (in the library's sense, at the vertex-and-edge-list encoding) then $\mathsf{PH} = \Sigma_2^p$. Interactive-machine characterizations, round collapse $\mathsf{AM}[k] = \mathsf{AM}$ and private-to-public-coin transformations are not formalized. [9, 10, 19, 21, 50][7, Section 8.2]

`Cslib.Complexity.MA`
`Cslib.Complexity.AM`
`Cslib.Complexity.NP_subset_MA`
`Cslib.Complexity.BPP_subset_MA`
`Cslib.Complexity.PolyTimeReducible.mem_MA`
`Cslib.Complexity.PolyTimeReducible.mem_AM`
`Cslib.Complexity.mem_MA_iff_exists_forall_errorExponent`
`Cslib.Complexity.mem_AM_iff_forall_errorExponent`
`Cslib.Complexity.MA_subset_AM`
`Cslib.Complexity.MA_subset_SBP`
`Cslib.Complexity.MA_subset_SigmaP_two`
`Cslib.Complexity.AM_subset_PiP_two`
`Cslib.Complexity.MA_subset_SigmaP_two_inter_PiP_two`
`Cslib.BitString.LinearHash.isPairwiseIndependentHash_affineHash`
`Cslib.Complexity.mem_AM_of_card_certifiedSet`
`Cslib.Complexity.SBP_subset_AM`
`Cslib.Complexity.GraphNonIsomorphism.PublicCoin.card_pairs`
`Cslib.Complexity.GraphNonIsomorphism.mem_AM`
`Cslib.Complexity.mem_AM_of_exists`
`Cslib.Complexity.PH_eq_SigmaP_two_of_coNP_subset_AM`
`Cslib.Complexity.SigmaP_two_eq_AM_of_coNP_subset_AM`
`Cslib.Complexity.PH_eq_SigmaP_two_of_NPComplete_GraphIsomorphism`

4.3 Toda's theorem

$$\mathsf{PH} \subseteq \mathsf{P}^{\#\mathsf{P}}.$$

Here #P counts witnesses of polynomial-time verifiers. The target class uses a polynomial-time oracle program answering decision problems via bits of one such counting function. The program is fixed before the oracle is chosen and must work for every oracle language agreeing on valid encoded queries; off-code answers are arbitrary. The target class has no separate query-count parameter. The nondeterministic-machine characterization of #P remains separate unfinished work.[124]

`Cslib.Complexity.PH_subset_PWithSharpPOracle`

4.4 Toda's theorem with a PP oracle

$$\mathsf{PH} \subseteq \mathsf{P}^{\#\mathsf{P}} = \mathsf{P}^{\mathsf{PP}}.$$

P^{PP} uses one polynomial-time oracle program, fixed before the oracle, correct against every oracle language agreeing with some PP problem on its encoded instances; off-code answers are arbitrary and no fixed query-count bound is imposed. It contains P^A for every $A \in \mathsf{PP}$. For $f \in \#\mathsf{P}$, $\{(b,k) \mid k \le f(b)\} \in \mathsf{PP}$ via GapP, and each #P bit request is answered by binary search with polynomially many PP queries, substituted into the program by open oracle substitution.

For the reverse inclusion, let $c(b)$ count the accepting strings of a PP verifier using $p(n)$ random bits, where n is the encoded input length. The function $h(b) = 2c(b) + (2^{p(n)} - 1)$ is in #P, and its zero-based bit $p(n)+1$ is one exactly when $2^{p(n)} < 2c(b)$. This includes zero random bits and rejects ties. Each PP query word w is replaced by the encoded pair $(w, p(|w|)+1)$, requesting that single bit of h without decoding w. The resulting deterministic program is fixed before the bit oracle is chosen and works for every oracle answering h on valid pair codes, with arbitrary answers elsewhere. Open substitution certifies uniform polynomial time, including the cost of constructing requests and reading replies; it is not merely a query bound. The direct shifted-count proof implements the standard equivalence of PP and counting oracles.[39, 124]

`Cslib.Complexity.PWithOracle_subset_PWithPPOracle`
`Cslib.Complexity.setOf_le_mem_PP`
`Cslib.Complexity.PWithSharpPOracle_subset_PWithPPOracle`
`Cslib.Complexity.PH_subset_PWithPPOracle`
`Cslib.Complexity.exists_SharpP_testBit_of_mem_PP`
`Cslib.Complexity.PWithPPOracle_eq_PWithSharpPOracle`

4.5 Easy oracles and the Valiant–Vazirani collapse

For an oracle language $A \in \mathsf{P}$, $\mathsf{P}^A = \mathsf{P}$, $\mathsf{NP}^A = \mathsf{NP}$ and $\mathsf{RP}^A = \mathsf{RP}$, and RP relative to a promise problem in PromiseP (under a decodable encoding) is RP. Hence, by Valiant–Vazirani, if PromiseUSAT (yes: uniquely satisfiable; no: unsatisfiable CNF formulas) has a polynomial-time solver then NP = RP for every encoding; so does USAT $\in$ P, a weaker form since UNSAT reduces to USAT.[7, 18, 131]

`Cslib.Complexity.ComputableInPolyTimeWithOracle.computableTidilyInPolyTime`
`Cslib.Complexity.PWithOracle_eq_P`
`Cslib.Complexity.NPWithOracle_eq_NP`
`Cslib.Complexity.RPWithOracle_eq_RP`
`Cslib.Complexity.RPWithPromiseOracle_eq_RP`
`Cslib.Complexity.NP_eq_RP_of_PromiseUSAT_mem_PromiseP`
`Cslib.Complexity.NP_eq_RP_of_USAT_mem_P`

4.6 Adleman's theorem

$$\mathsf{BPP} \subseteq \mathsf{P/poly}.$$

Every bounded-error polynomial-time language has a nonuniform family of polynomial-size De Morgan circuits. The result also covers encoded problems by taking the preimage of a bit-string language. It does not provide a uniform algorithm to choose the successful random strings or construct the circuits. [2, 7]

`Cslib.Circuits.Boolean.PPoly.exists_of_mem_BPP`

4.7 Derandomization from average-case hardness

Suppose a language L belongs to $\mathsf{E} = \mathsf{DTIME}(2^{O(n)})$ and there is a positive integer c such that, for every sufficiently large n, every De Morgan circuit of size at most $s_n = 2^{\lfloor n/c \rfloor}$ agrees with L on at most $1/2 + 1/s_n$ of the n-bit inputs. Then $\mathsf{P} = \mathsf{BPP}$, uniformly for encoded problems. The Nisan–Wigderson generator and exhaustive seed search have certified polynomial time. The next entry obtains the hypothesis from worst-case exponential circuit hardness.[94]

`Cslib.Complexity.P_eq_BPP_of_isAverageCaseHard`

4.8 Derandomization from worst-case hardness

Suppose a language L belongs to $\mathsf{E} = \mathsf{DTIME}(2^{O(n)})$ (on bit words) and there is a real $\gamma > 0$ such that, for every sufficiently large n, the De Morgan circuit complexity of L on n-bit inputs (fan-in two, every gate counted) is at least $2^{\gamma n}$. Then $\mathsf{P} = \mathsf{BPP}$ for every encoding of every type of instances. The hardness is converted to average-case hardness by the Sudan–Trevisan–Vadhan encoding: at input length N, with $M = \lceil 40/\gamma \rceil + 1$ and $b = \lfloor N/(4(M+1)) \rfloor$, the slice of L at length Mb, read as M blocks of b bits, is encoded by a Reed–Muller code over the explicit field $\mathbb{F}_{2^{4b}} = \mathbb{F}_2[X]/(g)$, with g the least irreducible polynomial of degree $4b$, concatenated with the Hadamard code; the remaining input bits are ignored. The encoded language lies in E whenever L does, with the field arithmetic and low-degree extension certified by the time calculus, and under the hardness hypothesis it is average-case hard for circuits of size $2^{\lfloor N/c \rfloor}$, $c = 200(\lceil 40/\gamma \rceil + 2)$, at every sufficiently large N; the previous result then applies. The hardness is assumed at all large lengths, as by Impagliazzo and Wigderson; the infinitely-often variant and the $\mathsf{EXP} \not\subseteq \mathsf{P/poly}$ consequence of Babai, Fortnow, Nisan and Wigderson are not formalized.[62, 122]

`Cslib.Complexity.P_eq_BPP_of_eventually_two_pow_le_complexity`
`Cslib.HardnessEncoding.encodedLanguage_mem_E`
`Cslib.HardnessEncoding.eventually_isAverageCaseHard_encodedSlice`
`Cslib.HardnessEncoding.encodedProgram_eq`
`Cslib.BinaryField.card`

4.9 Counting algebra and the GapP characterization of PP

For each fixed encoding, #P is closed under pointwise addition and multiplication, and GapP under addition, subtraction and multiplication; the constant natural-valued and integer-valued functions belong to the respective classes. Both classes are also closed under precomposition by a polynomial-time function between encoded instance types. The resulting count is exactly the original count on the mapped instance; no padding factor is introduced. Moreover,

$$A \in \mathsf{PP} \iff \exists g \in \mathsf{GapP}\ \forall x, \quad x \in A \iff g(x) > 0.$$

Here #P counts accepting certificates of a deterministic polynomial-time verifier of polynomial certificate length; GapP consists of differences of two such counts. These are verifier characterizations. The corresponding nondeterministic-machine characterizations remain unfinished. [37, 129]

`Cslib.Complexity.SharpP.subsemiring`
`Cslib.Complexity.GapP.subring`
`Cslib.Complexity.mem_PP_iff_exists_GapP`
`Cslib.Complexity.SharpP.comp_mem`
`Cslib.Complexity.GapP.comp_mem`

Polynomial-time functions with binary natural output belong to #P; those with binary integer output belong to GapP. The bound is on output bit length, so the numeric values may be exponential in input length. [39, Theorem 3.5]

`Cslib.Complexity.SharpP.of_polyTime`
`Cslib.Complexity.GapP.of_polyTime`

Uniform aggregation also permits input-dependent bounds. If $r(x)$ is computed in polynomial time with unary output and $g \in \mathsf{GapP}$, then $x \mapsto g(x)^{r(x)}$ is in GapP, including signed values and exponent zero. For a jointly uniform counting function $f(x, w)$ in #P or GapP on the encoded pair, the function

$$x \longmapsto \sum_{w \in \{0,1\}^{r(x)}} f(x, w)$$

belongs to the same class. Unary output ensures that the number of repeated factors and the word length are polynomially bounded; no polynomial bound on the numeric value of the resulting count is asserted. Variable certificate lengths are padded with false bits to preserve counts exactly. The signed-product verifier uses $r(x)(p_+(|x|) + p_-(|x|))$ certificate bits before final padding, where p_+ and p_- are the two original counting-certificate polynomials. For $f \in \#\mathsf{P}$, the same unary exponent condition gives $x \mapsto f(x)^{r(x)} \in \#\mathsf{P}$: an all-rows verifier uses exactly $r(x)p(|x|)$ certificate bits before padding, including zero rows and zero row width. More generally, if $f(x, i)$ is jointly in #P (respectively GapP) under the pair encoding with unary index i, and $r(x)$ is polynomial-time with unary output, then $x \mapsto \prod_{i<r(x)} f(x, i)$ belongs to the same class. For a #P certificate polynomial p, the construction pads each indexed certificate to width $p(2|x| + 1 + r(x))$ and uses $r(x)$ times that width before final padding. The empty product is one; zero factors and signed GapP factors are allowed. The GapP proof expands over Boolean choices of positive and negative counts, using the uniform #P product and an efficiently computed parity sign. This supplies a counting-algebra foundation; oracle lowness is not established. These uniform verifier closure results support the gap amplification below. [37]

`Cslib.Complexity.SharpP.of_polyTime_length`
`Cslib.Complexity.SharpP.sum_bits_mem`
`Cslib.Complexity.GapP.sum_bits_mem`
`Cslib.Complexity.GapP.pow_mem_polyTime`
`Cslib.Complexity.SharpP.pow_mem_polyTime`
`Cslib.Complexity.SharpP.prod_range_mem`
`Cslib.Complexity.GapP.prod_range_mem`

4.10 Relativized counting and majority verifiers

For every language oracle $A \subseteq \{0,1\}^*$ and fixed injective input encoding, $\#\mathsf{P}^A$ counts accepted Boolean witnesses of exactly $p(|x|)$ bits using a deterministic polynomial-time oracle verifier; GapP^A consists of differences of two such counts. The verifier uses one fixed word-query program, realized by a finite machine under one polynomial clock against every stateful word oracle, charging request construction and reply access. Only correctness of its Boolean output is specialized to the fixed, stateless language oracle A. The class PP^A uses the same verifier model and accepts exactly when strictly more than half the witnesses accept; ties reject. Then

$$L \in \mathsf{PP}^A \quad \Longleftrightarrow \quad \exists g \in \mathsf{GapP}^A \; \forall x, \quad x \in L \longleftrightarrow g(x) > 0.$$

The reverse construction pads two certificates of lengths $p(|x|)$ and $q(|x|)$ with false tails to length $(p+q)(|x|)$, then uses one selector bit; zero lengths are allowed. Each ordinary class is contained in its relativized counterpart by ignoring the oracle. If $A \in \mathsf{P}$ on words encoded as themselves, certified oracle simulation gives $\#\mathsf{P}^A = \#\mathsf{P}$, $\mathsf{GapP}^A = \mathsf{GapP}$ and $\mathsf{PP}^A = \mathsf{PP}$. These are verifier statements; nondeterministic/probabilistic oracle-machine characterizations and approximate-counting lowness remain separate. [7, 37, 39]

`Cslib.Complexity.SharpP_subset_SharpPWithOracle`
`Cslib.Complexity.GapP_subset_GapPWithOracle`
`Cslib.Complexity.SharpPWithOracle_eq_SharpP`
`Cslib.Complexity.GapPWithOracle_eq_GapP`
`Cslib.Complexity.PP_subset_PPWithOracle`
`Cslib.Complexity.PPWithOracle_eq_PP`
`Cslib.Complexity.mem_PPWithOracle_iff_exists_GapPWithOracle`

4.11 Boolean closure and unambiguous polynomial time

For every fixed injective encoding, P, NP and coNP are closed under union and intersection. The odd-acceptance class $\oplus$P is closed under union, intersection and symmetric difference. The verifier class UP satisfies

$$\mathsf{P} \subseteq \mathsf{UP} \subseteq \mathsf{NP} \cap \oplus\mathsf{P}.$$

Its verifier has at most one accepting certificate of the specified polynomial length on every input, and exactly one on members. Equivalently, a #P function bounded by one is one precisely on the language. UP is closed under polynomial-time many-one reductions. This establishes the verifier formulation; an equivalence with unambiguous nondeterministic machines is not asserted.[7, 98] The complement class coUP contains P and is contained in both coNP and $\oplus$P; it also inherits polynomial-time reduction closure. These are elementary consequences of the complement convention and the corresponding UP containments.

`Cslib.Complexity.P_subset_coUP`
`Cslib.Complexity.coUP_subset_coNP`
`Cslib.Complexity.coUP_subset_ParityP`
`Cslib.Complexity.inter_mem_P`
`Cslib.Complexity.union_mem_NP`
`Cslib.Complexity.inter_mem_coNP`
`Cslib.Complexity.symmDiff_mem_ParityP`
`Cslib.Complexity.P_subset_UP`
`Cslib.Complexity.UP_subset_NP`
`Cslib.Complexity.UP_subset_ParityP`
`Cslib.Complexity.PolyTimeReducible.mem_UP`

The unique-solution class US instead asks that the accepting count equal one exactly on members; no-instances may have zero or multiple witnesses. We prove $\mathsf{UP} \cup \mathsf{coNP} \subseteq \mathsf{US} \subseteq \mathsf{C}_{=}\mathsf{P}$, intersection closure, and polynomial-time many-one reduction closure. The definition follows Blass–Gurevich; these containments are elementary count identities. Unique-SAT completeness is recorded in the counting-completeness entry.[18]

`Cslib.Complexity.UP_subset_US`
`Cslib.Complexity.coNP_subset_US`
`Cslib.Complexity.US_subset_CeqP`
`Cslib.Complexity.inter_mem_US`
`Cslib.Complexity.PolyTimeReducible.mem_US`

4.12 Exact counting and PP

For the same verifier-based counting functions, the class $\mathsf{C_{=}P}$ is characterized by

$$A \in \mathsf{C_{=}P} \iff \exists g \in \mathsf{GapP}\ \forall x,\quad x \in A \iff g(x) = 0.$$

Equivalently, membership is equality of two #P functions. This class is closed under union, intersection and polynomial-time many-one reductions, and satisfies

$$\mathsf{coNP} \subseteq \mathsf{C_{=}P} \subseteq \mathsf{PP}.$$

The complement class $\mathsf{coC_{=}P}$ consists of nonzero sets of GapP functions and satisfies $\mathsf{NP} \subseteq \mathsf{coC_{=}P} \subseteq \mathsf{PP}$. It inherits union, intersection and reduction closure. This does not assert $\mathsf{C_{=}P} = \mathsf{coC_{=}P}$ or a counting-machine characterization. [37][39, Classifications 4.5, 4.8–4.9 and Corollaries 4.10–4.11]

`Cslib.Complexity.mem_coCeqP_iff_exists_GapP`
`Cslib.Complexity.NP_subset_coCeqP`
`Cslib.Complexity.coCeqP_subset_PP`
`Cslib.Complexity.mem_CeqP_iff_exists_SharpP`
`Cslib.Complexity.inter_mem_CeqP`
`Cslib.Complexity.coNP_subset_CeqP`
`Cslib.Complexity.CeqP_subset_PP`
`Cslib.Complexity.PolyTimeReducible.mem_CeqP`

4.13 Bounded ambiguity and small gaps

For each injective input encoding, FewP consists of languages recognized by positive #P counts bounded on every input by a polynomial in encoded input length. The certificate-length polynomial and ambiguity-bound polynomial are separate. SPP consists of languages whose characteristic function is in GapP: its gap is exactly one on members and zero on nonmembers. Then

$$\mathsf{UP} \subseteq \mathsf{FewP} \subseteq \mathsf{NP}, \qquad \mathsf{UP} \cup \mathsf{coUP} \subseteq \mathsf{SPP} \subseteq \mathsf{C_{=}P} \cap \mathsf{coC_{=}P}.$$

FewP is closed under union, intersection and polynomial-time many-one reductions; SPP has these closures and complement closure. The elementary proofs reuse the count algebra: FewP union adds ambiguity bounds and intersection multiplies them; SPP complement subtracts its gap from one and intersection multiplies gaps. These are verifier-based results; neither a machine characterization nor the stronger FewP–SPP containment is established by this entry; it is the next entry. The FewP definition follows Allender–Rubinstein and the explicit acceptance-set formulation of Borchert–Hemaspaandra–Rothe; the SPP definition and UP inclusion follow Fenner–Fortnow–Kurtz and Fortnow's survey. [5][22, Section 1] [37][39, Classification 4.4 and Corollary 4.11]

`Cslib.Complexity.UP_subset_FewP`
`Cslib.Complexity.FewP_subset_NP`
`Cslib.Complexity.PolyTimeReducible.mem_FewP`
`Cslib.Complexity.UP_subset_SPP`
`Cslib.Complexity.coUP_subset_SPP`
`Cslib.Complexity.compl_mem_SPP`
`Cslib.Complexity.SPP_subset_CeqP`
`Cslib.Complexity.SPP_subset_coCeqP`

4.14 FewP is in SPP

$$\mathsf{FewP} \subseteq \mathsf{SPP}.$$

With at most $q(n)$ accepted certificates, the increasing listings of accepted certificate sets of even and of odd size in $q(n)$ flagged slots are counted by polynomial-time verifiers ($E, O \in \#\mathsf{P}$), and $1 + O - E \in \mathsf{GapP}$ is the characteristic function; the count is an induction on slots. These are the verifier-based classes. [37, 76]

`Cslib.Complexity.SortedSubsets.signedSum_eq`
`Cslib.Complexity.FewP_subset_SPP`

4.15 Modular counting

For every fixed natural modulus $k \geq 2$, $\mathsf{Mod}_k\mathsf{P}$ consists of languages with a $\#\mathsf{P}$ witness count f such that

$$x \in A \quad \iff \quad f(x) \not\equiv 0 \pmod{k}.$$

The modulus is a class parameter, not an input. We prove $\mathsf{P} \subseteq \mathsf{UP} \subseteq \mathsf{Mod}_k\mathsf{P}$, $\mathsf{Mod}_2\mathsf{P} = \oplus\mathsf{P}$, and closure under polynomial-time many-one reductions. For prime k, the class is also closed under complement, union and intersection. The complement proof applies Fermat's little theorem to the count $1 + (k-1)f(x)^{k-1}$; intersection uses multiplication and primality. No corresponding composite-modulus Boolean closure is asserted. The total Lean definition also covers the degenerate moduli: at zero it equals NP, and at one only the empty language belongs. The standard $k \geq 2$ convention and nonzero-residue acceptance follow Fortnow's Classification 4.7; the closure proofs are elementary algebraic consequences of the existing counting APIs. [39, Classification 4.7][99]

`Cslib.Complexity.UP_subset_ModP`
`Cslib.Complexity.ModP_two`
`Cslib.Complexity.PolyTimeReducible.mem_ModP`
`Cslib.Complexity.compl_mem_ModP`
`Cslib.Complexity.inter_mem_ModP`
`Cslib.Complexity.union_mem_ModP`

The complement class $\mathsf{coMod}_k\mathsf{P}$ uses zero residues of a $\#\mathsf{P}$ count. It is closed under polynomial-time many-one reductions for every modulus, and equals $\mathsf{Mod}_k\mathsf{P}$ for prime k.

`Cslib.Complexity.mem_coModP_iff_exists_SharpP`
`Cslib.Complexity.PolyTimeReducible.mem_coModP`
`Cslib.Complexity.coModP_eq_ModP`

4.16 Wide counting gaps

For every fixed injective encoding,

$$\mathsf{UP} \cup \mathsf{coUP} \cup \mathsf{FewP} \subseteq \mathsf{SPP} \subseteq \mathsf{LWPP} \subseteq \mathsf{WPP} \subseteq \mathsf{C_{=}P} \cap \mathsf{coC_{=}P}.$$

In WPP there are a GapP function g and a polynomial-time, binary-integer function f, nonzero on every input, with $g(x) = f(x)$ on members and $g(x) = 0$ on nonmembers. LWPP restricts the target to $f(|x|)$, with f computable from unary length and nonzero on every natural argument, including zero. Negative and exponentially large targets are allowed. Both classes are closed under complement, union, intersection, set difference, symmetric difference, and polynomial-time many-one reductions between injectively encoded types. Furthermore, both classes satisfy finite multiple-target robustness: if $g \in \mathsf{GapP}$ is 0 outside A and on A takes a value in a fixed finite family $\{f_i\}_{i \in s}$ of nonzero polynomial-time binary-integer targets (depending on x for WPP or on unary length $|x|$ for LWPP), then $\prod_{i \in s} f_i - \prod_{i \in s}(f_i - g) \in \mathsf{GapP}$ witnesses membership with single nonzero target $\prod_{i \in s} f_i$. For LWPP, a nonzero unary-input target $f(k(x))$ at a polynomial-time unary-computable index $k(x) \leq p(|x|)$ is normalized to $\prod_{i \leq p(|x|)} f(i)$ by multiplying $g(x)$ by the binary-integer list product $\prod_{i \leq p(|x|),\, i \neq k(x)} f(i)$, yielding reduction closure when $k(x) = |r(x)|$. These verifier-based statements follow the definitions of Fenner–Fortnow–Kurtz and the finite-family and known-index cases of Hemaspaandra et al. Full polynomial-family robustness with an unknown accepting index is stated next. Counting-machine characterizations and oracle lowness are separate. [37][54, Definitions 2.4–2.5 and Section 3]

`Cslib.Complexity.SPP_subset_LWPP`
`Cslib.Complexity.LWPP_subset_WPP`
`Cslib.Complexity.FewP_subset_LWPP`
`Cslib.Complexity.FewP_subset_WPP`
`Cslib.Complexity.WPP_subset_CeqP`
`Cslib.Complexity.WPP_subset_coCeqP`
`Cslib.Complexity.compl_mem_WPP`
`Cslib.Complexity.inter_mem_WPP`
`Cslib.Complexity.union_mem_WPP`
`Cslib.Complexity.diff_mem_WPP`
`Cslib.Complexity.symmDiff_mem_WPP`
`Cslib.Complexity.PolyTimeReducible.mem_WPP`
`Cslib.Complexity.compl_mem_LWPP`
`Cslib.Complexity.inter_mem_LWPP`
`Cslib.Complexity.union_mem_LWPP`
`Cslib.Complexity.diff_mem_LWPP`
`Cslib.Complexity.symmDiff_mem_LWPP`
`Cslib.Complexity.mem_WPP_of_finset_targets`
`Cslib.Complexity.mem_LWPP_of_finset_targets`
`Cslib.Complexity.mem_LWPP_of_polyTime_index`
`Cslib.Complexity.PolyTimeReducible.mem_LWPP`
`Turing.MultiTapeTM.ComputableTidilyInPolyTime.intBin_prod`

4.17 Polynomial-family wide-gap robustness

For every fixed injective encoding, polynomial-size accepting and rejecting target families preserve the verifier-based wide-gap classes:

$$(\mathsf{Poly},\mathsf{Poly})\text{-}\mathsf{WPP} = \mathsf{WPP}, \qquad (\mathsf{Poly},\mathsf{Poly})\text{-}\mathsf{LWPP} = \mathsf{LWPP}.$$

Precisely, let $g \in \mathsf{GapP}$ and let $Y(x), R(x)$ be uniformly polynomial-time computable lists of signed binary integers. Suppose their sets of values are disjoint at each input and

$$x \in A \Rightarrow g(x) \in Y(x), \qquad x \notin A \Rightarrow g(x) \in R(x).$$

Then $A \in \mathsf{WPP}$. If both lists are computed only from unary input length, and are disjoint at every natural length including zero, then $A \in \mathsf{LWPP}$. The list-output bounds charge both the number of targets and their binary lengths. Neither target index needs to be computable. Original targets may be negative, zero, or repeated; zero is allowed on either side, provided the two families remain disjoint. An empty list excludes the corresponding language side at that input, and empty products have value one. Conversely, singleton accepting targets and rejection list [0] recover ordinary witnesses. The indexed forms use uniform FP functions $y(x,i), r(x,j)$, or $y(n,i), r(n,j)$ for LWPP, with unary indices and two independently chosen counts computed in polynomial time with unary output. Disjointness concerns the used index ranges. These forms include the standard positive polynomial-size families, after translating their one-based indices. The construction follows Hemaspaandra, Hemaspaandra, Spakowski and Watanabe: the new gap

$$h(x) = \prod_{r \in R(x)} (r - g(x))$$

vanishes on rejection. Its accepting targets are $\prod_{r\in R(x)}(r-y)$ for $y \in Y(x)$; disjointness makes these products nonzero. Uniform GapP closure for this product of differences and polynomial-time binary integer products reduce to the one-sided target theorem.[54, Definition 3.8, Corollary 3.11 and Theorem 3.12] The proof is Appendix A of the full version, arXiv:1711.01250.

In particular, fixing the rejection list to [0] yields $\mathsf{Poly}\text{-}\mathsf{WPP} = \mathsf{WPP}$ and $\mathsf{Poly}\text{-}\mathsf{LWPP} = \mathsf{LWPP}$, with nonzero accepting targets and an unknown accepting index.[54, Theorems 3.1 and 3.7] The underlying counting algebra credits Fenner–Fortnow–Kurtz.[37] Oracle lowness and counting-machine characterizations remain separate.

`Cslib.Complexity.mem_WPP_iff_exists_disjoint_targets`
`Cslib.Complexity.mem_LWPP_iff_exists_disjoint_targets`
`Cslib.Complexity.mem_WPP_of_disjoint_indexed_targets`
`Cslib.Complexity.mem_LWPP_of_disjoint_indexed_targets`
`Cslib.Complexity.mem_WPP_iff_exists_polyTime_targets`
`Cslib.Complexity.mem_LWPP_iff_exists_polyTime_targets`
`Cslib.Complexity.GapP.prod_sub_mem`

4.18 Uniform bounded wide-gap closure

Fix an injective instance encoding, with encoded length $|x|$. Let A be a single jointly uniform language on pairs (x, i), using the pair encoding with unary i, and let $r(x)$ be computable in polynomial time with unary output. For either $\mathcal{C} = \mathsf{WPP}$ or $\mathcal{C} = \mathsf{LWPP}$,

$$A \in \mathcal{C} \quad \Longrightarrow \quad \begin{array}{l} \{x : \forall i < r(x),\ (x, i) \in A\} \in \mathcal{C}, \\ \{x : \exists i < r(x),\ (x, i) \in A\} \in \mathcal{C}. \end{array}$$

Membership of every slice separately is not the hypothesis: the joint language must have one uniform witness. The count may grow with the input and may differ between inputs of the same length. Empty ranges give the full language for conjunction and the empty language for disjunction.

For WPP, multiply the indexed gaps and their nonzero binary FP targets. For LWPP, the joint encoded length is exactly $2|x| + 1 + i$. If f is its nonzero length-only target and a fixed $p \in \mathbb{N}[X]$ bounds $r(x)$ by $p(n)$ at length n, all possible accepting product targets lie in the length-only polynomial family

$$T(n, k) = \prod_{i<k} f(2n + 1 + i), \qquad 0 \le k \le p(n).$$

Every target is nonzero, including $T(n, 0) = 1$. Polynomial-family robustness normalizes this family even though the selected index $k = r(x)$ is input-dependent. Complement closure gives the existential result. This is a formalized deduction from Fenner–Fortnow–Kurtz counting algebra and the polynomial-family robustness of Hemaspaandra, Hemaspaandra, Spakowski and Watanabe. [37][54, Theorem 3.1] The classes retain the verifier-based model; these results do not establish adaptive oracle closure or PP-lowness.

`Cslib.Complexity.forall_lt_mem_WPP`
`Cslib.Complexity.exists_lt_mem_WPP`
`Cslib.Complexity.forall_lt_mem_LWPP`
`Cslib.Complexity.exists_lt_mem_LWPP`

4.19 Small bounded-error and almost-wide counting

For every fixed injective encoding,

$$\mathsf{NP} \cup \mathsf{BPP} \subseteq \mathsf{SBP} \subseteq \mathsf{PP}, \qquad \mathsf{SPP} \cup \mathsf{BPP} \subseteq \mathsf{AWPP} \subseteq \mathsf{PP}.$$

SBP has a verifier-based #P function f and a polynomial-time function k with binary natural output such that $f(x) \geq k(x)$ on members and $2f(x) < k(x)$ on nonmembers. The nonnegative threshold may be zero on members; requiring it to be positive on every input gives the same class. AWPP has a verifier-based GapP function g and $p \in \mathbb{N}[X]$ with $0 \leq g(x) \leq 2^{p(|x|)}$ on every input, and

$$x \in A \Rightarrow 3g(x) \geq 2 \cdot 2^{p(|x|)}, \qquad x \notin A \Rightarrow 3g(x) \leq 2^{p(|x|)}.$$

Both classes are closed under polynomial-time many-one reductions between injectively encoded types. AWPP is closed under complement; SBP is closed under union. The AWPP reduction proof permits an input-dependent, unary polynomial-time exponent and pads it to a polynomial in encoded input length. These are elementary consequences of the existing counting algebra and BPP amplification. The definitions follow Watson's formulation of the class introduced by Böhler–Glaßer–Meister, and Fenner's constant-error characterization of AWPP.[19] [133, Definition 2][36] Counting-machine characterizations and PP-lowness remain outstanding. The following entries add amplification and Boolean closure.

`Cslib.Complexity.NP_subset_SBP`
`Cslib.Complexity.BPP_subset_SBP`
`Cslib.Complexity.SBP_subset_PP`
`Cslib.Complexity.mem_SBP_iff_exists_pos_threshold`
`Cslib.Complexity.union_mem_SBP`
`Cslib.Complexity.PolyTimeReducible.mem_SBP`
`Cslib.Complexity.BPP_subset_AWPP`
`Cslib.Complexity.SPP_subset_AWPP`
`Cslib.Complexity.AWPP_subset_PP`
`Cslib.Complexity.compl_mem_AWPP`
`Cslib.Complexity.mem_AWPP_of_polyTime_exponent`
`Cslib.Complexity.PolyTimeReducible.mem_AWPP`

4.20 Wide gaps, amplification and Boolean closure

For every fixed injective encoding,

$$\mathsf{LWPP} \subseteq \mathsf{WPP} \subseteq \mathsf{AWPP}.$$

The WPP target may be signed, input-dependent and exponentially large: it is a nonzero, binary-output polynomial-time integer function, with the GapP witness equal to that target on members and zero on nonmembers. The classical containment is credited to Fenner–Fortnow–Kurtz–Li; this formalization squares the target and uses binary length to obtain a dyadic normalization.[38, Section 6.1]

Every AWPP language has a GapP witness g and $p \in \mathbb{N}[X]$ such that, writing $D(x) = 2^{p(|x|)}$, one has $0 \le g(x) \le D(x)$ and

$$x \in A \Rightarrow 7D(x) \le 8g(x), \qquad x \notin A \Rightarrow 8g(x) \le D(x).$$

Three fixed rounds of the majority-of-three polynomial give this error bound; the denominator exponent is multiplied by 27. This is a fixed-size instance of Fenner's amplification method.[36]

AWPP is closed under intersection, union, difference and symmetric difference of two languages over the same encoding, as well as complement.

For SBP, every exponent $r(x)$ polynomially bounded in encoded input length admits one #P witness f and a positive binary-output polynomial-time natural threshold k with

$$x \in A \Rightarrow k(x) \le f(x), \qquad x \notin A \Rightarrow 2^{r(x)} f(x) < k(x).$$

A fixed length polynomial q dominating r gives $q(|x|) + 1$ repetitions, raising both the original count and threshold to that power. The request need not be computable. This uniform consequence of Watson's powering argument strengthens the multiplicative gap; it is not a normalized additive error bound.[133, Sections 1.1 and 3] Also $A \in \mathsf{SBP}$ and $B \in \mathsf{BPP}$ imply $A \cap B \in \mathsf{SBP}$, using the existing input-dependent BPP error reduction and a dyadic bound on #P counts. This does not establish intersection closure for two SBP languages. All statements use verifier-based #P and GapP; their nondeterministic-machine characterizations remain open formalization work.

`Cslib.Complexity.WPP_subset_AWPP`
`Cslib.Complexity.LWPP_subset_AWPP`
`Cslib.Complexity.exists_AWPP_gap_eighth`
`Cslib.Complexity.inter_mem_AWPP`
`Cslib.Complexity.union_mem_AWPP`
`Cslib.Complexity.diff_mem_AWPP`
`Cslib.Complexity.symmDiff_mem_AWPP`
`Cslib.Complexity.exists_SBP_gap_pow_polyBounded`
`Cslib.Complexity.inter_mem_SBP_of_mem_BPP`

4.21 Uniform AWPP error reduction

Fix an injective bit-string encoding; $|x|$ denotes its encoded length. For every $A \in \mathsf{AWPP}$ and every $r : X \to \mathbb{N}$ polynomially bounded in encoded input length, there exist $G \in \mathsf{GapP}$ and $P \in \mathbb{N}[X]$ such that, writing $D(x) = 2^{P(|x|)}$, for every input x,

$$0 \le G(x) \le D(x), \qquad \begin{aligned} x \in A &\Rightarrow 2^{r(x)}(D(x) - G(x)) \le D(x), \\ x \notin A &\Rightarrow 2^{r(x)} G(x) \le D(x). \end{aligned}$$

The requested rate r need not be computable: the proof chooses a fixed $q \in \mathbb{N}[X]$ dominating it. Given an original denominator exponent p, strict majority of $12q(|x|)$ trials yields the explicit exponent $P = 12qp$. Zero exponents and a denominator of one are allowed. Uniform GapP word sums and unary powers construct the numerator; a finite counting identity transfers the majority tail bound without assuming an efficient sampler for the original GapP value. The constant-error characterization and error reduction are credited to Fenner; the factor 12 comes from the formalized majority estimate used for BPP. Here GapP retains the verifier-based model; counting-machine characterizations and PP-lowness are not established. [36][7, Section 7.4.1]

`Cslib.Complexity.AWPPAmplification.majorityPolynomial_mem`
`Cslib.Complexity.AWPPAmplification.amplify`
`Cslib.Complexity.exists_AWPP_gap_errorExponent`

5 Cryptographic constructions

The one-way functions and pseudorandom objects below use the library's bit-string security experiments and machine-certified efficiency predicates. Security is asymptotic against uniform strict polynomial-time probabilistic adversaries. In the one-wayness game, x is uniform among n-bit strings; the inverter receives unary n and $f(x)$ and succeeds with any preimage, even one of a different length. The function need not preserve length.

5.1 Goldreich–Levin hard-core predicate

For every one-way function f, the inner product $\langle x, r\rangle$ over $\mathbb{F}_2$ is a hard-core predicate for the padded map $(x, r) \mapsto (f(x), r)$. The formal construction splits a seed into equal halves and retains an optional spare bit, covering every seed length. The predicate is polynomial-time computable; every uniform PPT predictor has negligible advantage over a fair guess. [46]

`Cslib.Crypto.GoldreichLevin.padded_hardCore`

5.2 Pseudorandom generators from arbitrary one-way functions

Every uniform one-way function implies a deterministic polynomial-time pseudorandom generator expanding each n-bit seed to $n + 1$ bits, secure against uniform strict PPT distinguishers. The proof follows Holenstein's pseudoentropy route to the Håstad–Impagliazzo–Levin–Luby theorem. OWF existence is the sole cryptographic assumption: no entropy estimate or nonuniform advice is supplied as an extra hypothesis. The result does not claim optimized construction exponents.[56, 59]

`Cslib.Crypto.OneWay.exists_pseudorandomGenerator`

5.3 Polynomial stretch with quantitative resource bounds

Let G be a one-bit-stretch PRG and let $k : \mathbb{N} \to \mathbb{N}$ be positive and polynomial-time computable with unary input and output. Repeatedly expanding the retained n-bit seed yields a PRG with output length $n + k(n)$, secure against the same uniform strict PPT distinguishers. In particular, one-bit stretch suffices for any strictly larger, unary-polynomial-time computable target length. [12] The construction also has a separate machine-level resource certificate. Suppose G has time/workspace bounds t_G, s_G, the count computation has bounds t_k, s_k, and every round's seed u_i satisfies $s_G(u_i) \le Q(u)$ for initial seed u. Writing $n = |u|$ and $k = k(n)$, the certified bounds are

$$\begin{aligned} \text{time} &= O\left(t_k(n) + n + k + \sum_{i<k} (t_G(u_i) + n + 1) + 1\right), \\ \text{workspace} &= O(s_k(n) + Q(u) + n + k + 1)\,. \end{aligned}$$

The exact one-bit output-length invariant is required. The workspace bound includes the final reversal buffer; a small streaming state alone does not give an output-length-independent bound for the full construction.

`Cslib.Crypto.PRGStretch.pseudorandomGenerator`
`Cslib.Crypto.PRGStretch.computableTidilyInTimeAndSpace_stretch`

5.4 GGM and adaptive pseudorandom functions

Every uniform pseudorandom generator gives an efficiently evaluable pseudorandom function family by polynomial stretching and the GGM tree construction. Security holds against uniform adaptive strict polynomial-time oracle adversaries, not just fixed query lists. At parameter n, keys, valid queries and answers have n bits; both games answer malformed queries with the empty word. Adversaries receive the unary parameter and an empty auxiliary word. Consequently every uniform one-way function implies such a PRF. These are conditional security implications, not assertions that one-way functions exist.[47, 56]

`Cslib.Crypto.PseudorandomGenerator.exists_pseudorandomFunction`
`Cslib.Crypto.OneWay.exists_pseudorandomFunction`

5.5 Generators imply one-way functions

A secure PRG G whose output length $\ell(n)$ satisfies $\ell(n) \ge 2n$ for every seed length n is itself one-way, including against inverters that return preimages of different lengths. More generally, G is one-way if ℓ is injective and $n \mapsto 2^{-(\ell(n)-n)}$ is negligible. An arbitrary PRG with positive stretch yields a one-way function after truncation and stretch amplification; the claim in this case is existence of a possibly different function. Together with HILL, this gives

$$\text{a one-way function exists} \iff \text{a pseudorandom generator exists.}$$

All security notions are the uniform strict-PPT notions specified above. Neither side's unconditional existence is asserted.[59, 73]

`Cslib.Crypto.PseudorandomGenerator.oneWay`
`Cslib.Crypto.PseudorandomGenerator.oneWay_of_injective`
`Cslib.Crypto.exists_oneWay_iff_exists_pseudorandomGenerator`

5.6 One-way functions separate P and BPP from NP

For f computable in polynomial time, the preimage-prefix problem $\{(1^n, y, u) \mid \exists x \sqsupseteq u,\ |x| = n,\ f(x) = y\}$ is in NP. If it is in P, bit-by-bit search gives a deterministic polynomial-time inverter; if it is in BPP, amplification to error $2^{-(L+1)}$ on instances of encoded length L and a union bound over the n queries of the search give a uniform strict PPT inverter succeeding with probability at least $1/2$. Hence a uniform one-way function puts its preimage-prefix problem outside P and BPP, and, by Cook–Levin and closure of P and BPP under polynomial-time reductions, SAT $\notin$ P and SAT $\notin$ BPP. The existence of one-way functions is not asserted.[44, 73]

`Cslib.Crypto.preimagePrefix_mem_NP`
`Cslib.Crypto.OneWay.P_ne_NP`
`Cslib.Crypto.OneWay.SAT_notMem_P`
`Cslib.Crypto.OneWay.not_NP_subset_BPP`
`Cslib.Crypto.OneWay.SAT_notMem_BPP`
`Cslib.Complexity.PolyTimeReducible.mem_BPP`

5.7 Kabanets–Cai: MCSP in BPP rules out one-way functions

If MCSP, with instances encoded by the 2^m-bit truth table followed by the size threshold in binary and De Morgan circuit size counting all gates, is in BPP, then no pseudorandom function family secure against uniform adaptive strict oracle-PPT adversaries exists, hence no uniform one-way function exists; in particular this holds if MCSP is in P. The hypothesis is MCSP $\in$ BPP at all thresholds; the distinguisher reads the truth table of the oracle on the 2^m words with $n - m$ leading zeros, $m = (c+2)\lceil \log_2 n \rceil$, and runs the decider only at threshold $\lfloor 2^m/(5m) \rfloor$: restrictions of a polynomial-time family have circuits of size at most $c(3n+2)^c + 2$ uniformly in the key, while for $m \geq 16$ at most $2^{2^m/4}$ of the 2^{2^m} truth tables have circuits with at most $\lfloor 2^m/(5m) \rfloor$ gates. The advantage is at least $1/4$ for all large n.[47, 66]

`Cslib.Crypto.not_pseudorandomFunction_of_MCSP_mem_BPP`
`Cslib.Crypto.not_oneWay_of_MCSP_mem_BPP`
`Cslib.Crypto.not_oneWay_of_MCSP_mem_P`
`Cslib.Crypto.KabanetsCai.isOraclePPT_mcspDistinguisher`
`Cslib.Crypto.KabanetsCai.exists_complexity_restriction_le`

6 Circuit depth, lower bounds and barriers

Circuit classes in this section are nonuniform. Unless a different cost is specified, native De Morgan circuits charge constants, NOT, AND and OR gates, with wiring and fanout free. Formula leaf count and formula depth are separate measures; unbounded fan-in bases are identified in their entries.

6.1 Shannon–Lupanov bounds and the MCSP window

Count all NOT, AND, OR and constant gates. For every $\varepsilon > 0$, all sufficiently large n, and every n-input truth table, MCSP answers yes at any integer threshold $s \geq (1+\varepsilon)2^n/n$. For every sufficiently large n, some truth table needs more than $\lfloor 2^n/n \rfloor$ gates. A finite counting bound makes the scarcity of small circuits explicit: for $n \geq 16$, at most $2^{\lfloor 2^n/4 \rfloor}$ of the 2^{2^n} truth tables have a circuit of size at most $\lfloor 2^n/(5n) \rfloor$. These are synthesis and counting statements; they do not establish the computational hardness of deciding MCSP.[89, 114]

`Cslib.MCSP.eventually_mem`
`Cslib.MCSP.eventually_exists_not_mem`
`Cslib.MCSP.ncard_setOf_mem_le`

6.2 Barrington's theorem

Nonuniform NC^1 is exactly the class of languages recognized by polynomial-length permutation branching programs of width five. An equivalent formulation uses programs over the alternating group A_5. This is a class equality with the library's nonuniform circuit families; it does not assert a uniform-family characterization.[13, 14]

`Cslib.Circuits.Boolean.NC_one_eq_polyLength_perm_five`
`Cslib.Circuits.Boolean.NC_one_eq_polyLength_alternatingGroup`

6.3 Spira's formula balancing theorem

Every De Morgan formula with $L \geq 1$ input leaves has an equivalent formula with at most $4\lceil \log_2 L \rceil$ AND/OR gates on each input-to-output path. Constants contribute zero leaves; when $L = 0$, the formal bound is zero. Negations have zero cost in this depth measure. The theorem certifies the depth of the balanced formula, not a machine runtime for constructing it.[120]

`Cslib.Circuits.Boolean.exists_formula_depthWith_le_four_mul_clog`

6.4 Khrapchenko's bound and exact parity formula size

Let a De Morgan formula be true on A and false on B, and let $E(A, B)$ be the pairs in $A \times B$ at Hamming distance one. Its input-leaf count L (constants contribute zero) satisfies

$$|E(A, B)|^2 \leq |A|\,|B|\,L.$$

In particular, parity on n bits requires at least n^2 leaves; for $n = 2^k$ this is attained. Yablonskii's construction gives, for $n = 2^k + r$ with $0 \leq r \leq 2^k$, a parity formula with at most $2^k(2^k + 3r)$ leaves; for every n there is one with $8L \leq 9n^2$. These are formula leaf bounds, not bounds on circuits with shared subcomputations. [65, 74, 75, 135]

`Cslib.Circuits.Formula.sq_card_hammingEdges_le`
`Cslib.Circuits.Boolean.isLeast_leafSize_parity_two_pow`
`Cslib.Circuits.Boolean.exists_formula_parity_two_pow_add`
`Cslib.Circuits.Boolean.exists_formula_parity_eight_mul_leafSize_le`

6.5 Parity lower bounds by random restrictions

A depth-d circuit of unbounded fan-in AND/OR gates and negations computing parity on n bits has at least 2^s gates whenever

$$s \geq 2, \qquad s(16s + 1)^d \leq n.$$

Thus the finite theorem gives an exponential lower bound at every fixed depth. Depth counts negations and size counts every gate. The exponent obtained is of order $n^{1/(d+1)}$, weaker than the sharp classical Håstad exponent $n^{1/(d-1)}$, which the next entry proves.[4, 41, 53]

`Cslib.Circuits.Boolean.Unbounded.two_pow_le_size_of_computes_parity`

6.6 Håstad's theorem with the sharp exponent

Let C be a circuit of negations and unbounded fan-in AND/OR gates of AND/OR depth at most d (the largest number of AND and OR gates on a path; negations are free, and constants are nullary AND/OR gates) computing parity on n bits. Then C has at least 2^s AND/OR gates whenever

$$s \geq 2, \qquad 17s(16s+1)^{d-2} \leq n$$

($d-2$ truncated at 0). Hence for $d \geq 2$ it has at least 2^s gates whenever $(17s)^{d-1} \leq n$, and at least $\frac{1}{2}\,2^{n^{1/(d-1)}/17}$ gates once $n \geq 34^{d-1}$. The first bound also holds when depth counts negations, and for formulas. With the Lecomte–Ramakrishnan upper bound $2^{O(n^{1/(d-1)})} + n$ already in the library, the exponent $1/(d-1)$ is optimal. The proof is the staged switching argument with stages indexed by AND/OR depth, a first stage switching 1-DNFs (keeping a $1/17$ fraction of the variables), and no final stage: a DNF whose terms are shorter than the number of free variables cannot compute restricted parity.[7, 53, 65, 81]

`Cslib.Circuits.Boolean.Unbounded.two_pow_le_card_andOr_of_computes_parity`
`Cslib.Circuits.Boolean.Unbounded.two_pow_le_size_of_computes_parity_of_depthWith_le`
`Cslib.Circuits.Boolean.Unbounded.two_pow_le_size_of_computes_parity_of_pow_le`
`Cslib.Circuits.Boolean.Unbounded.two_rpow_le_two_mul_size_of_computes_parity`
`Cslib.Circuits.Boolean.Unbounded.two_pow_le_size_of_computes_parity_of_depth_le`
`Cslib.Circuits.Boolean.Unbounded.two_pow_le_size_of_formula_computes_parity_of_depthWith_le`

6.7 Parity is not first-order definable

Let L be a relational vocabulary with finitely many relation symbols, $P \in L$ unary, and N_n an arbitrary L-structure on $\{0, \ldots, n-1\}$ for each n (numerical relations such as order or BIT, chosen separately per size). Write $N_n[P{:=}S]$ for N_n with P reinterpreted as S. No first-order sentence φ of L satisfies, for all large n and all $S \subseteq \{0, \ldots, n-1\}$, $N_n[P{:=}S] \models \varphi \iff |S|$ is even; that is, parity is not in FO[Arb]. In particular no sentence defines "$|P|$ is even" on all finite L-structures with at least n_0 elements. Constant symbols are not supported. The proof substitutes input bits and constants into the constant-depth polynomial-size formula deciding φ and applies the random-restriction parity bound for formulas above.[4, 41, 60]

`Cslib.Circuits.Boolean.ModelChecking.not_eventually_forall_realize_expansion_iff_even`
`Cslib.Circuits.Boolean.ModelChecking.not_forall_realize_iff_even_card`

6.8 Razborov–Smolensky lower bounds

For every odd prime p, parity is not in nonuniform $\mathrm{AC}^0[p]$: adding unbounded fan-in MOD_p gates to constant-depth polynomial-size AND/OR/NOT circuits does not suffice. Quantitatively, a depth-d circuit computing parity has size at least $2^{\ell+1}/5$ whenever $\ell \geq 1$ and

$$100\big(((p-1)\ell)^d + 1\big)^2 \leq n + 1.$$

The prime and depth are fixed in the class separation.[102, 119]

`Cslib.Circuits.Boolean.Smolensky.two_mul_two_pow_le_size_of_computes_parity`
`Cslib.Circuits.Boolean.Smolensky.parity_notMem_ACOMod`

6.9 Razborov's monotone clique bound

Let $p, w \geq 2$, $w+1 \leq k \leq n$, $p \leq n$, and $2w^2 \leq k-1$, and put

$$B = (p-1)^w w!, \quad A = B^2 \binom{n-w-1}{k-w-1}, \quad N = (2B+B^2)(w^2)^p (k-1)^{n-p}.$$

Every monotone AND/OR circuit for k-clique on n vertices has more than s gates if $As < \binom{n}{k}$ and $2Ns < (k-1)^n$. This states the formalized finite approximation-method bound with its parameters; no unrestricted Boolean-circuit lower bound follows from it.[101]

`Cslib.Circuits.Monotone.lt_size_of_computes_clique`

6.10 The natural-proofs barrier

Assume a family of length-doubling generators of polynomial circuit size and a fixed integer $d \geq 1$ such that, for all sufficiently large seed lengths k and every m with $m^d \leq k$, no circuit of size at most 2^m distinguishes its output from uniform with advantage greater than 2^{-m}. Then no P/poly-natural property is useful against P/poly. Here natural means containing a property of density at least $2^{-O(n)}$ whose truth-table language has circuits polynomial in the table length 2^n; useful means eventually excluding the slices of every P/poly language. The security assumption is stronger than ordinary polynomial-time PRG security. [47, 105]

`Cslib.Circuits.Boolean.not_isUseful_PPoly_of_isNatural_of_generator`

6.11 Exponentially small correlation with quadratic forms

There is a nonconstructively chosen family of bilinear Boolean functions h_n with the following property. Put

$$c_* = 1 + \frac{\pi}{6 \arccos((1+2\sqrt{2})/4)} \approx 2.78125.$$

For every real $c < c_*$, there is $\gamma > 0$ such that for all sufficiently large n, every Boolean circuit of fan-in at most two, over any Boolean basis, with at most cn positive-arity gates has

$$\left| 2 \Pr_{x \in \{0,1\}^n} [C(x) = h_n(x)] - 1 \right| \leq 2^{-\gamma n}.$$

The input is uniform, constants are not charged, and the cutoff and rate are independent of the circuit. The forms are obtained by choosing submatrix-robust matrices; no efficient construction or uniform complexity-class membership is proved. The exact coefficient above is the library's frontier-method result; related correlation-bound sources are credited separately.[28, 51, 64]

`Cslib.Circuits.Frontier.exists_eventually_abs_correlation_hardForm_le_of_lt`

6.12 Weighted-threshold activity and capacity bounds

Consider real-weight threshold gates, and count as input-reading a gate with a nonzero effective coefficient on an original input; repeated wires are aggregated before testing this condition. If a circuit computes inner product modulo two on two m-bit blocks, with its output taken from a gate, let q be its number of input-reading gates and e the maximum number of those gates outputting true on one input. Then

$$2^m \leq \left(\sum_{j=0}^{e} \binom{q}{j} \right)^2.$$

The same inequality holds with q counting all gates and e their maximum simultaneous activity.[1]

`Cslib.Circuits.Boolean.WeightedThreshold.two_pow_le_sum_choose_sq_of_circuit_hadamardCode`
`Cslib.Circuits.Boolean.WeightedThreshold.two_pow_le_sum_choose_gateActivity_sq_of_circuit_hadamardCode`

A separate capacity bound applies to any fixed Boolean readout of a threshold program's gate values computing f on n bits. Suppose that, across every partition of the input coordinates, every rectangle contained in either fiber of f has a side with fewer than K assignments. If q gates read original inputs in the effective sense above, then

$$2^n < 4K^2\,4^q.$$

This statement requires the rectangle hypothesis on both fibers; it is not an unconditional lower bound for every Boolean function.[108]
`Cslib.Circuits.Boolean.WeightedThreshold.two_pow_lt_mul_four_pow_inputGates`

6.13 Exact parity formula depth at powers of two

Let $d(f)$ be minimum De Morgan formula depth, counting binary AND/OR gates and making negations free. For $n \geq 1$,

$$\lceil \log_2(n^2) \rceil \leq d(\mathrm{PARITY}_n) \leq 2\lceil \log_2 n \rceil, \qquad d(\mathrm{PARITY}_{2^k}) = 2k.$$

The lower bound comes from formula leaf count, and the upper bound from the parity construction. This does not assert the exact minimum for every non-power-of-two arity.[65, 75, 135]
`Cslib.Circuits.Boolean.clog_sq_le_formulaDepth_parity`
`Cslib.Circuits.Boolean.formulaDepth_parity_le_two_mul_clog`
`Cslib.Circuits.Boolean.formulaDepth_parity_two_pow`

6.14 Quadratic completeness of semifilter covers

Let $A \subseteq U$ and let $B_1, \ldots, B_n \subseteq U$ be supplied input sets. A semifilter on $U \setminus A$ is an upward-closed family containing the whole complement and excluding the empty set. It is above a point $a \in A$ if it contains every $B_i \cap (U \setminus A)$ for which $a \in B_i$. Suppose a list of t pairs of complement subsets defeats every semifilter above any point of A: for some listed pair it accepts both components but not their intersection. Then there is an acyclic set circuit computing A from the supplied sets with at most t^2 AND gates. Binary unions and both constants are free, and fanout shares computed sets. Consequently the minimum AND cost is at most the square of the minimum unrestricted pair-cover length (with value ∞ if no cover exists). The construction works even for infinite U, empty input families, and trivial targets: each point's t pair coordinates stabilize within t rounds. Selecting free unions uses classical subset tests; no effective compiler running-time bound is claimed. This is the native free-constant version of Cavalar–Oliveira's quadratic completeness theorem, following the Fusion method. [26]
`Cslib.Circuits.Fusion.PairCover.cost_toCircuit`
`Cslib.Circuits.Fusion.PairCover.eval_toCircuit`
`Cslib.Circuits.Fusion.ecomplexityFrom_le_pairCoverComplexity_sq`

6.15 Graph Fusion transfer and exact inequality complexity

Let $V = \{0,1\}^n$, let $G \subseteq V \times V$, and supply every row and column as a free source. A semifilter on G^c accepts the full set, rejects the empty set, and is upward closed. It is above an edge when it accepts the restricted row and column containing that edge. A cover defeats every such witness with a pair of accepted sets whose intersection is rejected; $\rho(G)$ is the minimum number of pairs. Representing a Boolean input by two n-bit blocks gives

$$\rho(G) \le \rho(f_G; \text{positive and negative literals}).$$

The inequality also holds when both sides restrict to semi-ultrafilters (every set or its complement is accepted), and includes trivial graphs. For $G_{\neq} = \{(u,v) : u \neq v\}$,

$$\rho_{\text{can}}(G_{\neq}) = \rho(G_{\neq}) = D_{\cap}(G_{\neq}) = n.$$

Canonical semifilters are the upward closures of a complementary row and column, both nonempty. Here $D_{\cap}$ counts only binary intersections in native acyclic circuits: unions, empty/full constants, rows, columns, wiring and fanout are free. The equality includes $n = 0$, when the graph is empty and all three costs are zero. This is a graph intersection bound, not a total-gate count.[26]

`Cslib.Circuits.Fusion.Graph.pairCoverComplexity_le_literals`
`Cslib.Circuits.Fusion.Graph.inequality_complexities`

6.16 The fixed-distribution Fusion counting barrier

Let $A \subseteq \{0,1\}^n$, let $v \in A$, and let F be a semifilter on $U = \{0,1\}^n \setminus A$. Suppose F accepts $\{u \in U : u_i = v_i\}$ for every coordinate i. There is an explicit distribution on pairs (S,T) of subsets of U, depending only on A, which covers every such (v,F) with probability at least $1/(8n+8)$. Covering means $F(S)$ and $F(T)$ hold but $F(S \cap T)$ does not. The sampler independently and uniformly chooses a coordinate or an extra complement case, a Boolean polarity, and a full truth table.

More generally, for any finite family (v_w, F_w) satisfying these hypotheses and nonnegative real weights μ_w, there exists a pair covering total weight at least

$$\frac{\sum_w \mu_w}{8n+8}.$$

If the weights sum to one and every pair covers weight at most d, then $d > 0$ and $1/d \le 8n+8$. This bounds the fixed-distribution counting certificate, not minimum cover size or circuit complexity. The hitting pair can depend on the weighted family; the sampling distribution does not. The theorem applies even if the weights are chosen after seeing a circuit, provided the capacity bound holds for every pair. It does not bound certificates whose capacity constraint ranges only over that circuit's gate pairs.

The proof adapts Oliveira's exposition of Razborov's argument through successive random truth-table cylinders. It needs only acceptance of matching literals, not exact agreement on the opposite literals, and proves the sampling lemma rather than assuming it. No efficient sampling bound is asserted. [96, 103]

`Cslib.Circuits.Fusion.le_booleanPairSample_expect`
`Cslib.Circuits.Fusion.exists_pair_weight_ge`
`Cslib.Circuits.Fusion.fixedDistribution_counting_bound`

6.17 Linear cover complexity of random bipartite graphs

Fix n supplied subsets of a finite carrier. The number of targets admitting an unrestricted semifilter cover with at most t pairs is at most $2^{(2t+1)(n+t+1)}$. Each cover is encoded by the inclusion flags used in quadratic closure propagation, and decoding is proved to recover its target; no incompressibility or encoding hypothesis remains. For a uniform labelled graph $G \subseteq [N] \times [N]$, take all $2N$ rows and columns as supplied sets and write $\rho(G)$ for unrestricted cover complexity. Every graph has $\rho(G) \le N$, and

$$\Pr[\rho(G) \le t] \le \frac{2^{(2t+1)(2N+t+1)}}{2^{N^2}}.$$

In particular, for every $N \ge 16$,

$$\Pr[\lfloor N/16 \rfloor < \rho(G) \le N] \ge 1 - 2^{-N}.$$

The distribution is uniform over all 2^{N^2} bipartite graphs, including empty and complete graphs. This is a finite quantitative form of the linear high-probability bound of Cavalar–Oliveira, not an explicit hard-graph construction.[26]

`Cslib.Circuits.Fusion.card_small_covers_le`
`Cslib.Circuits.Fusion.Graph.pairCoverComplexity_le_card`
`Cslib.Circuits.Fusion.Graph.smallCoverProbability_le`
`Cslib.Circuits.Fusion.Graph.random_cover_complexity`

6.18 Semi-ultrafilter characterization of universal verifiers

Let $f : \{0,1\}^n \to \{0,1\}$ and write $\rho_{\mathrm{ultra}}(f)$ for pair-cover complexity with respect to its positive and negative literals, restricting witnesses to semi-ultrafilters. A semi-ultrafilter accepts every set or its complement, possibly both. Let $C_{\forall,\wedge}(f) \in \mathbb{N} \cup \{\infty\}$ be the infimum of the native AND cost over all finite auxiliary counts a, all verifiers $g : \{0,1\}^{n+a} \to \{0,1\}$ with $f(x) = \forall y\, g(x,y)$, and all native circuits implementing g from the positive and negative literals of all $n + a$ inputs. Then, in the extended natural numbers,

$$\rho_{\mathrm{ultra}}(f) \le C_{\forall,\wedge}(f) \le 5\rho_{\mathrm{ultra}}(f).$$

More concretely, every universal verifier with t binary AND gates yields a pair cover of length exactly t. Conversely, every semi-ultrafilter pair cover of length t yields a universal verifier with exactly $3t$ auxiliary bits and $5t$ AND gates. Its bits label the two components and intersection of every pair; consistent labels extend to a semi-ultrafilter, while every false input supplies a consistent, pair-preserving certificate.

Constants, OR gates, supplied literals, wiring and fanout are free. Both constructions include zero auxiliary bits, empty covers, constant functions and $n = 0$. The measure concerns circuit cost, not verifier runtime or the cost of finding a cover or constructing the circuit. This proves the linear conondeterministic characterization credited to Karchmer; the explicit factor five comes from the grouped certificate construction formalized here. [26]

`Cslib.Circuits.Fusion.exists_pairCover_forallGuess`
`Cslib.Circuits.Fusion.PairCover.exists_forallGuess`
`Cslib.Circuits.Fusion.conondeterministicAndComplexity_bounds`

7 Proof complexity

7.1 Cook–Reckhow characterization

NP equals coNP for every encoded instance type if and only if UNSAT has a polynomially bounded Cook–Reckhow proof system: a polynomial-time function from bit strings onto unsatisfiable CNF formulas, with a preimage of polynomial length for every such formula. Polynomial boundedness asserts that short proofs exist; it does not give a polynomial-time procedure for finding them. [31]

`Cslib.Complexity.ProofSystem.NP_eq_coNP_iff_exists_polynomiallyBounded`

7.2 Resolution size–width tradeoff

If a CNF formula has initial clause width at most k, at most N variables, and a resolution refutation with at most s lines, it has a refutation of some integer width w satisfying

$$w \le k + 2\sqrt{2N \ln s} + 2.$$

Thus a large minimum refutation width forces a large refutation size. The statement concerns proof certificates, not the runtime of a resolution or DPLL solver.[16]

`Cslib.Logic.PL.CNF.exists_refutableInWidth_le_of_refutableInSize`

7.3 Tseitin formulas are hard for resolution

Let G be a finite multigraph on V with edge set E (self-loops and parallel edges allowed) and $\chi : V \to \{0, 1\}$ a charge. The Tseitin formula $T(G, \chi)$ has a variable per edge and, for each vertex v, at most $2^{d(v)}$ clauses of width $d(v)$ stating that the parity of the non-loop edges at v is $\chi(v)$ ($d(v)$ = number of non-loop edges at v). It is unsatisfiable whenever $\sum_v \chi(v)$ is odd; on a connected G it is satisfiable exactly when $\sum_v \chi(v)$ is even. Let $e(G)$ be the least number of edges between a set S with $|V|/3 \le |S| \le 2|V|/3$ and its complement ($e(G) = 0$ if $|V| \le 1$). On a connected G every resolution refutation of $T(G, \chi)$ has width (distinct literals in a line) at least $e(G)$; if moreover every vertex has at most k non-loop edges, $|E| \le N$ and $k + 2 \le e(G)$, every refutation has at least $\exp((e(G) - k - 2)^2/(8N))$ lines. For the multigraph of a rotation map, $e(G) \ge h(G)|V|/3$ with h the edge expansion not normalized by the degree. Consequently, for every $n \ge 4392000$ there is an unsatisfiable CNF with at most $120n$ variables and $2^{120} \cdot n$ clauses of width at most 120 all of whose resolution refutations have at least $\exp(n/(960 \cdot 36000^2))$ lines. The formula is the Tseitin formula, with one charged vertex, of a 120-regular expander whose existence is proved non-constructively; the constants are far from optimal, and the statement concerns proof size, not solver runtime. [16, 65, 125, 126]

`Cslib.Logic.PL.CNF.not_satisfiable_tseitinFormula`
`Cslib.Logic.PL.CNF.satisfiable_tseitinFormula_iff`
`Cslib.Logic.PL.Tseitin.minBalancedCut_le_of_refutableInWidth`
`Cslib.Logic.PL.Tseitin.exp_le_of_refutableInSize`
`Cslib.RotationMap.edgeExpansion_mul_card_div_three_le_minBalancedCut`
`Cslib.Logic.PL.CNF.exists_forall_refutableInSize_exp_le`

8 Communication and formulas

8.1 Karchmer–Wigderson correspondence

For a Boolean function on $n > 0$ bits, minimum De Morgan formula depth, counting AND/OR gates and making negations free, equals deterministic communication complexity of the following relation: Alice receives a true input, Bob a false input, and they must output a coordinate on which the inputs differ. The equality uses the library's worst-case protocol depth and extended-natural complexity. [70]

`Cslib.Circuits.Boolean.formulaDepth_eq_ecomplexity`

8.2 Yao's minimax principle

For finite input spaces and a nonempty output space, the public-coin communication complexity of a relation with worst-case error at most ε equals the supremum, over input distributions, of the minimum deterministic communication cost with distributional error at most ε. Both directions are proved, including the minimax step; the result is not only the easy averaging lower bound. Costs are worst-case numbers of communicated bits.[78, 136]

`Cslib.Communication.PublicCoin.ecomplexity_eq_iSup_distributionalComplexity`

8.3 Newman's theorem

If each party's input space has cardinality at most 2^n, then for every $\delta > 0$, private-coin communication complexity at error $\varepsilon + \delta$ is at most public-coin complexity at error ε plus

$$\left\lceil \log_2 \left\lceil n/\delta^2 \right\rceil \right\rceil$$

bits. The formal expression uses natural ceiling and `Nat.clog`, including its zero-argument convention. The shared finite sample of protocols is chosen existentially, without a certified procedure to construct it.[91]

`Cslib.Communication.PrivateCoin.ecomplexity_le_add_clog`

8.4 Randomized communication complexity of disjointness

Deciding whether two n-bit strings have disjoint supports needs $\Omega(n)$ bits with bounded error: for every n, the public-coin communication complexity R of disjointness with worst-case error $1/3$ satisfies $n \le 2^{16}R + 2^{12}$, and the same holds for private coins; with error $\varepsilon \le 1/64$, $n \le 2^9 R_\varepsilon + 2^{12}$. Public-coin protocols are arbitrary distributions over deterministic protocols, with cost the largest depth in the support. The proof is Razborov's corruption argument with a variant of his distribution, using all subsets of the parts rather than fixed-size ones: a uniformly random partition (i, T_x, T_y) of $2m + 1$ points into a point and two m-sets, uniformly random subsets $x \subseteq T_x$, $y \subseteq T_y$, and with probability $1/2$ the point i added to both sets. On every rectangle the intersecting pairs carry at least $1/18$ of the mass of the disjoint pairs, up to $(27/32)^r/2$ when $48r \le m+1$. Error $1/3$ is reduced to $1/64$ by majority votes of 72 independent runs. Costs are worst-case numbers of communicated bits, and the constants are not optimized. [69, 78, 104]

`Cslib.Communication.PublicCoin.le_mul_ecomplexityFun_disjointness_add`
`Cslib.Communication.PrivateCoin.le_mul_ecomplexityFun_disjointness_add`
`Cslib.Communication.PublicCoin.le_mul_ecomplexityFun_disjointness_add_of_le`
`Cslib.Communication.Disjointness.corrupted_hardDist`

The majority amplification holds for every Boolean function: at error $(4\varepsilon(1-\varepsilon))^r$, public-coin complexity is at most $2r$ times the complexity at error $\varepsilon \le 1/2$.

`Cslib.Communication.PublicCoin.ecomplexityFun_le_two_mul_mul`

8.5 Monotone depth of bipartite perfect matching

Every monotone formula (AND/OR gates of fan-in two, constants free) deciding whether a subgraph of $K_{n,n}$, given by its n^2 edge indicators, has a perfect matching has depth at least $(n - 2^{14})/2^{13}$, and the monotone Karchmer–Wigderson game of bipartite perfect matching needs that many bits. The proof is a randomized reduction from the disjointness bound of the preceding entry at error $1/64$: Alice's set becomes a perfect matching on three-vertex gadgets, Bob's set a Hall-violating graph, and after a uniformly random relabelling of both sides the edge found reveals an intersection with probability at least $1/2$, by a symmetry of both graphs; for $0 < m$, $2m \le n$ and $k > 0$, public-coin disjointness of m bits at error 2^{-k} costs at most $k(c + 1)$ bits given a game protocol of c bits. Raz and Wigderson proved the bound for perfect matching in general graphs; this entry covers the bipartite case only, and the constants are not optimized.[78, 100]

`Cslib.Circuits.Monotone.le_mul_eformulaDepth_bipartitePerfectMatching_add`
`Cslib.Circuits.le_mul_ecomplexity_bipartitePerfectMatching_add`
`Cslib.Circuits.ecomplexityFun_disjointness_le_mul_ecomplexity_bipartitePerfectMatching`
`Cslib.Circuits.bipartitePerfectMatching`

8.6 Finite-set equality and inner-product rank

Equality and inequality on a finite set X with $|X| \ge 2$ each have deterministic communication complexity exactly $\lceil \log_2 |X| \rceil + 1$. The protocol model labels leaves by the answer, so the transcript determines the output; the bound includes the bit announcing the comparison result.

For inner product modulo two on two n-bit inputs, the sign matrix has rank 2^n and the $0/1$ communication matrix has rank $2^n - 1$ over any field of characteristic different from two. A deterministic protocol computing it has at least $2^n - 1$ leaves labelled true. These are exact-protocol and algebraic bounds, not randomized communication lower bounds.[78]

`Cslib.Communication.ecomplexityFun_equality`
`Cslib.Communication.ecomplexityFun_not_equality`
`Cslib.Communication.rank_signMatrix_dotProduct`
`Cslib.Communication.rank_communicationMatrix_dotProduct`
`Cslib.Communication.two_pow_sub_one_le_card_true_leaves_of_run_eq_dotProduct`

9 Boolean analysis and expander walks

9.1 Hypercontractivity on the Boolean cube

For a real-valued function on a finite Boolean cube with uniform probability measure, $1 \le p \le 2 \le q < \infty$, and $0 \le \rho \le \sqrt{(p-1)/(q-1)}$, the noise operator satisfies

$$\|T_\rho f\|_q \le \|f\|_p.$$

This includes the $(2, 4)$ inequality at $\rho = 1/\sqrt{3}$. The indexed theorem covers the range $p \le 2 \le q$ and finite real exponents; it is not a claim of all hypercontractive parameter regimes.[20, 95]

`Cslib.BitString.lpNorm_noiseOp_le_lpNorm`

9.2 The Kahn–Kalai–Linial theorem and Talagrand's inequality

Let f be a Boolean function on a finite cube with $n \geq 1$ coordinates, read as the sign $F = (-1)^f$, with influences $\mathrm{Inf}_i[F]$ (the probability that flipping bit i changes f), total influence $I[F] = \sum_i \mathrm{Inf}_i[F]$, and variance $\mathrm{Var}[F]$, all for the uniform measure. Then some coordinate has

$$\mathrm{Inf}_i[F] \geq \frac{\mathrm{Var}[F]\,\log n}{12\,n}.$$

This follows from the edge-isoperimetric form: if every influence is at most δ, then $\mathrm{Var}[F]\log(1/\delta) \leq 6\,I[F]$. If every influence is less than one, $\mathrm{Var}[F] \leq 6\sum_i \mathrm{Inf}_i[F]/\log(1/\mathrm{Inf}_i[F])$ (Talagrand), and for every f, $\mathrm{Var}[F] \leq 7\sum_i \mathrm{Inf}_i[F]/(1+\log(1/\mathrm{Inf}_i[F]))$; terms with $\mathrm{Inf}_i[F] = 0$ contribute 0. Logarithms are natural; constants are those of the formal proof, not optimized. The proof applies the library's $(4/3, 2)$-hypercontractive inequality to the derivatives D_iF.[68, 95, 123]

`Cslib.BitString.exists_expect_sub_expect_sq_mul_log_card_div_le_influence`
`Cslib.BitString.expect_sub_expect_sq_mul_log_inv_le_six_mul_totalInfluence`
`Cslib.BitString.expect_sub_expect_sq_le_six_mul_sum_influence_div_log`
`Cslib.BitString.expect_sub_expect_sq_le_seven_mul_sum_influence_div_one_add_log`

9.3 Friedgut's junta theorem

Let f be a Boolean function on a finite cube whose sign $F = (-1)^f$ has total influence $I[F] \leq K$, and let $\varepsilon > 0$. Then there are a set J of at most $\frac{4K^3}{9\varepsilon^2}\,81^{K/\varepsilon}$ coordinates and a Boolean function g depending only on the coordinates in J with $\Pr_x[f(x) \neq g(x)] \leq \varepsilon$; for $0 < \varepsilon \leq 1$ one may take $|J| \leq 3 \cdot 2^{8K/\varepsilon}$. In the proof, J is the set of coordinates of influence above an explicit threshold and g is the sign of the average of F over the other coordinates. The low-degree part of O'Donnell's spectral concentration statement (sets of size at most $I[F]/\varepsilon$) is not formalized.[40, 95]

`Cslib.BitString.exists_dependsOn_hammingDist_div_le_of_totalInfluence_le`
`Cslib.BitString.exists_card_le_three_mul_two_rpow_dependsOn_hammingDist_div_le`

9.4 Expander-walk hitting bound

Let a finite undirected regular graph be represented by a rotation map with nonempty vertex and port sets, and let λ be its nonconstant spectral operator-norm bound. Start a walk at a uniform vertex and choose each port uniformly. If each set B_i has density at most $0 \leq \mu \leq 1$, then

$$\Pr[v_i \in B_i \text{ for every } 0 \leq i \leq t] \leq \mu\big(\mu + \lambda(1-\mu)\big)^t.$$

The sets may vary with time. This is a finite probability bound; machine-level error reduction and extractor evaluation costs are separate obligations. [3, 67, 127]

`Cslib.RotationMap.dens_filter_forall_walkAt_mem_le`

9.5 The Nisan–Szegedy bound on relevant coordinates

If a Boolean function on a finite cube, read as the sign $(-1)^{f(x)}$, has no nonzero Fourier coefficients on sets of more than d coordinates, then at most $d\,2^{d-1}$ coordinates are relevant: flipping an irrelevant coordinate never changes the value. For $d = 0$ the function has no relevant coordinates; the formal natural exponent uses truncated subtraction. The Fourier coefficients use the uniform measure on the cube. [93, 95]

`Cslib.BitString.card_filter_exists_update_ne_le`

9.6 Fourier bounds for decision trees

Suppose a deterministic decision tree has structural depth d and L leaves. For any real-valued readout of its output bounded in absolute value by one, the sum of absolute Fourier coefficients is at most $L \le 2^d$. For a Boolean readout, written as signs, each coefficient is an integer multiple of $2^{-\max(d-1,0)}$, and at most $L\,2^{\max(d-1,0)}$ coefficients are nonzero. Repeated queries and unreachable branches are allowed; structural size only weakens these upper bounds. The Kushilevitz–Mansour entry below uses the spectral bound to learn decision trees.[77, 95]

`Cslib.DecisionTree.sum_abs_fourierCoeff_le_leafCount`
`Cslib.DecisionTree.exists_fourierCoeff_signVal_eq_intCast_div_two_pow_depth_sub_one`
`Cslib.DecisionTree.card_filter_fourierCoeff_signVal_ne_zero_le`

9.7 The Linial–Mansour–Nisan low-degree learning algorithm

A deterministic learner receives m labelled examples $(x_j, f(x_j))$ at independent uniform points of $\{0,1\}^n$; its error is $\Pr_x[h(x) \ne f(x)]$. Let $N = \#\{S \subseteq [n] : |S| \le t\} \le (n+1)^t$. The low-degree algorithm estimates each $\hat{f}(S)$, $|S| \le t$, empirically and outputs the sign of the truncated expansion. If $\varepsilon, \delta > 0$, every target has Fourier weight at most $\varepsilon/2$ above degree t, and $m \ge \frac{4N}{\varepsilon} \ln \frac{2N}{\delta}$, then the hypothesis has error at most ε with probability at least $1-\delta$; for $\varepsilon, \delta \in (0,1)$ such a learner is a PAC learner, in CSLib's sense, over the uniform realizable family. For circuits of NOT and unbounded fan-in AND/OR gates of depth at most d (every gate counted, negations included) with at most g gates, the degree $t = 2\max(8, 2s)(16s+1)^d - 1$, $s = \max(2, \lceil \log_2 \lceil 4g/\varepsilon \rceil \rceil)$, i.e. $t = O(\log(g/\varepsilon))^{d+1}$, suffices. For a circuit with negations only at the inputs and standard AND/OR depth d', this depth is at most $d'+1$ (an informal conversion). The exponent $d+1$ is weaker than LMN's d and than the sharper $O(\log(g/\varepsilon))^{d-1} \cdot \log(1/\varepsilon)$. Only the number of examples is certified; the $O(Nmt)$ arithmetic cost is not machine-certified, and the quasi-polynomial reading for $g = \mathrm{poly}(n)$ is not formalized. [55, 85, 95, 128]

`Cslib.MachineLearning.UniformLearning.lowDegreeLearner_learns`
`Cslib.MachineLearning.UniformLearning.Learns.isPACLearnerFor`
`Cslib.MachineLearning.UniformLearning.lowDegreeLearner_learns_lmnDegree`
`Cslib.Circuits.Boolean.Unbounded.sum_fourierCoeff_sq_le_of_computes_of_depth_le`
`Cslib.BitString.hammingDist_signRound_sum_mul_chi_div_le`

9.8 The Kushilevitz–Mansour algorithm with membership queries

A membership-query learner is, for each random seed, a decision tree whose variables are the points of $\{0,1\}^n$; its query count is the tree depth. The Kushilevitz–Mansour algorithm estimates the bucket weights $\sum_{U \cap J = S} \hat{f}(U)^2$ (O'Donnell's $\mathbf{W}^{S|\bar{J}}$), which equal $\mathbb{E}_{x,y,z}[f(x_J, z)\chi_S(x) f(y_J, z)\chi_S(y)]$, from m shared samples and runs the Goldreich–Levin bucket search with threshold $\theta^2/2$, $\theta = \varepsilon/(2L)$; on accurate estimates every U with $|\hat{f}(U)| \ge \theta$ is found and at most $4/\theta^2$ buckets survive per level. If $\varepsilon, \delta > 0$ and $m \ge \frac{512L^4}{\varepsilon^4} \ln \frac{2(n+2)2^n}{\delta}$, it learns every f with $\sum_S |\hat{f}(S)| \le L$ to error ε with probability at least $1-\delta$, using $m(2n+3)$ non-adaptive queries (informally, polynomially many in $n, L, 1/\varepsilon, \log(1/\delta)$); in particular this holds for decision trees with at most L leaves. Samples are shared across all buckets and the analysis takes a union bound over all 2^n buckets per level. Only the query count is certified; the running time of the search is not machine-certified. [6, 46, 77, 95]

`Cslib.MachineLearning.UniformLearning.kmLearner_learns`
`Cslib.MachineLearning.UniformLearning.kmLearner_learns_decisionTree`
`Cslib.MachineLearning.UniformLearning.kmLearner_queriesAtMost`
`Cslib.MachineLearning.UniformLearning.mul_card_bucketSearch_le_one`
`Cslib.BitString.bucketWeight_eq_expect_sample`
`Cslib.MachineLearning.UniformLearning.QueryLearner.learns_ofLearner_iff`

9.9 A degree-dependent obstruction to perfect expansion

For an undirected D-regular multigraph on $N \geq 2$ vertices, with $D \geq 1$, let λ be the norm of its normalized walk operator on mean-zero functions. Then

$$\frac{N - D}{D(N-1)} \leq \lambda^2.$$

In particular $\lambda \geq 1/\sqrt{2D}$ if $N \geq 2D$. The rotation-map model permits loops and parallel edges. The complete graph with one loop at each vertex has $D = N$ and $\lambda = 0$, attaining the first bound. This finite trace bound is not the asymptotic Alon–Boppana theorem. [57]

`Cslib.RotationMap.div_le_sq_spectralExpansion`
`Cslib.RotationMap.one_div_sqrt_two_mul_le_spectralExpansion`
`Cslib.RotationMap.spectralExpansion_complete`

10 Sensitivity and query complexity

For total Boolean functions on n bits, write s for maximum sensitivity, bs for maximum block sensitivity, C for certificate complexity, and D for deterministic decision-tree depth. A subscript b restricts a maximum to inputs with output b; it is zero if there are no such inputs.

10.1 Polynomial relations between query measures

For every total Boolean function f,

$$C(f) \leq s(f)\, bs(f) \leq bs(f)^2, \qquad D(f) \leq C(f)\, bs(f) \leq bs(f)^3.$$

The formal development also gives $C(f) \leq 2s(f)^5$ and $D(f) \leq 4s(f)^9$, using the existing sensitivity theorem and degree bounds. These are upper bounds for exact deterministic queries, with no promise on the input.[23, 58, 92, 93]

`Cslib.QueryComplexity.certificateComplexity_le_sensitivity_mul_blockSensitivity`
`Cslib.QueryComplexity.certificateComplexity_le_blockSensitivity_sq`
`Cslib.DecisionTree.complexity_le_certificateComplexity_mul_blockSensitivity`
`Cslib.DecisionTree.complexity_le_blockSensitivity_pow_three`
`Cslib.QueryComplexity.certificateComplexity_le_two_mul_sensitivity_pow_five`
`Cslib.DecisionTree.complexity_le_four_mul_sensitivity_pow_nine`

10.2 Monotone measures and symmetric evasiveness

For monotone f, $C(f) = bs(f) = s(f)$ and $D(f) \leq s_0(f)s_1(f) \leq s(f)^2$. Separately, every nonconstant symmetric Boolean function on n bits is evasive: $D(f) = n$. Symmetry means dependence only on Hamming weight. Thus, for Th_j^n, which accepts iff at least j bits are true, with $1 \leq j \leq n$,

$$D(\mathrm{Th}_j^n) = n, \qquad C(\mathrm{Th}_j^n) = bs(\mathrm{Th}_j^n) = s(\mathrm{Th}_j^n) = \max(j, n+1-j).$$

The equality of the three local-change and certificate measures requires monotonicity; it is not asserted for arbitrary functions. [23, 92]

`Cslib.QueryComplexity.certificateComplexity_eq_sensitivity_of_monotone`
`Cslib.QueryComplexity.blockSensitivity_eq_sensitivity_of_monotone`
`Cslib.DecisionTree.complexity_le_sensitivityFor_mul_sensitivityFor_of_monotone`
`Cslib.DecisionTree.complexity_eq_card_of_isSymmetric`
`Cslib.BitString.sensitivity_threshold`

10.3 Approximate degree of disjunction

Let $\widetilde{\deg}_{1/3}(f)$ be the least degree of a real polynomial approximating the 0/1 value of f to error at most $1/3$ at every Boolean input. For disjunction on n bits,

$$n \le 6\,\widetilde{\deg}_{1/3}(\mathrm{OR}_n)^2, \qquad \widetilde{\deg}_{1/3}(\mathrm{OR}_n) \le \lfloor\sqrt{2n}\rfloor + 1.$$

These explicit finite bounds establish the square-root order of growth. The upper bound uses a Chebyshev polynomial; no optimal leading constant is claimed. [23, 93]

`Cslib.QueryComplexity.card_le_six_mul_approxDegree_threshold_one_right_sq`
`Cslib.QueryComplexity.approxDegree_threshold_one_right_le_sqrt`

10.4 Superquadratic separation of sensitivity and block sensitivity

There is a fixed real exponent $\alpha > 2$ and a family of nonconstant Boolean functions F_m on positive, varying arities such that $F_m(0) = 0$ and

$$s(F_m)^\alpha \le bs(F_m, 0), \qquad bs(F_m, 0) \longrightarrow \infty.$$

Here 0 is the all-false input, and $bs(f, 0)$ is block sensitivity at that specific input. In particular, for every real c there exists a nonconstant f with $c\,s(f)^2 < bs(f)$, so no universal quadratic upper bound holds. The exponent is existential, not an assertion for every $\alpha > 2$. The linked construction ports OpenAI's formalization through the earlier complexitylib development and connects it to the library's public measures. [97, 112]

`Cslib.QueryComplexity.exists_sensitivity_rpow_le_blockSensitivityAt`
`Cslib.QueryComplexity.exists_mul_sensitivity_sq_lt_blockSensitivity`

11 Finite combinatorial inequalities

11.1 Discrete Loomis–Whitney

For a finite set A in a product of $n \ge 2$ finite coordinate spaces, let $\pi_i(A)$ be its projection obtained by deleting coordinate i. Then

$$|A|^{n-1} \le \prod_{i=1}^{n} |\pi_i(A)|.$$

The coordinate spaces may differ and A may be empty. This is the discrete cardinality inequality, obtained from the counting form of Shearer's lemma; no continuous-volume theorem is asserted here. [29, 87]

`Cslib.Probability.PMF.card_pow_le_prod_card_image_restrict_erase`

11.2 A lower bound for sunflower thresholds

For every $p \ge 2$ and $w \ge 0$, there is a family of exactly $(p-1)^w$ distinct sets, each of size w, containing no sunflower with p petals. The construction uses the graphs of all functions from a w-element set to a $(p-1)$-element set. A sunflower means distinct sets with all pairwise intersections equal to a common core. This lower bound need not be optimal: the six edges of two disjoint triangles are a family of two-element sets with no three-petal sunflower. [34]

`Cslib.exists_card_eq_pow_not_containsSunflower`
`Cslib.Sunflower.card_twoTriangles`
`Cslib.Sunflower.card_of_mem_twoTriangles`
`Cslib.Sunflower.not_containsSunflower_twoTriangles`

12 Reed–Solomon decoding

12.1 Berlekamp–Welch unique decoding

Let $\alpha_1, \dots, \alpha_n$ be distinct points of a field, and let w be a received word. If $2e + k \le n$, the decoder returns a polynomial P exactly when $\deg P < k$ and its evaluation word differs from w in at most e positions. It returns failure exactly when no such polynomial exists. The formal decoder chooses a solution of the key equations noncomputably; its correctness is proved, but polynomial decoding runtime is not certified.[52, 134]

`Cslib.Code.berlekampWelch_eq_some_iff`
`Cslib.Code.berlekampWelch_eq_none_iff`

12.2 Sudan's list-decoding guarantee

For distinct field evaluation points, $k \ge 2$, and a natural number D satisfying

$$2(k-1)n < (D+1)(D+2),$$

Sudan's decoder lists exactly the polynomials of degree below k agreeing with the received word at more than D positions. The number of listed polynomials is at most $\lfloor D/(k-1) \rfloor$. Interpolation and root containment are proved; the interpolant is chosen noncomputably, and no decoding runtime or Guruswami–Sudan multiplicity improvement is asserted.[52, 121]

`Cslib.Code.mem_sudan_iff_of_lt`
`Cslib.Code.card_sudan_mul_le`

13 Arithmetic circuits and simultaneous computation

13.1 Polynomial-size circuits for the determinant

For every commutative ring K and $n \in \mathbb{N}$, the generic $n \times n$ determinant has a division-free arithmetic circuit over K of size at most

$$3n^4 + 4.$$

Consequently its polynomial family belongs to VP over K. The same circuit evaluates determinants over every commutative K-algebra, with matrix entries supplied in row-major order. This is a gate-count assertion, not a certified uniform circuit-generation bound.[17, 130]

`Cslib.Circuits.Arithmetic.exists_circuit_det`
`Cslib.Circuits.Arithmetic.exists_circuit_det_eval`
`Cslib.Circuits.Arithmetic.detFamily_mem_VP`

13.2 Circuit mass production with leading coefficient $1/(1-\gamma)$

Fix real $0 \le \gamma < 1$ and $\varepsilon > 0$. For every sufficiently large n, every Boolean function f on n bits, and every integer $1 \le k \le 2^{\gamma n}$, one circuit computes f independently on k disjoint n-bit input blocks with nonconstant-gate cost at most

$$\left(\frac{1}{1-\gamma} + \varepsilon\right)\frac{2^n}{n}.$$

The cutoff depends on γ, ε, not on f, k. Here NOT, AND and OR each cost one and constants cost zero, unlike the total-gate convention used in other entries. The coefficient is optimal at $\gamma = 0$, where a single copy of most functions already needs $(1 - o(1))2^n/n$ nonconstant gates[114][65, Theorem 1.14]; no matching lower bound is claimed for $\gamma > 0$. The construction is nonuniform and supplies no certified runtime for choosing its tables.[110]

`Cslib.Circuits.Boolean.MassProduction.realFineMassProduction`

14 Circuits for iterated functions

14.1 Fixed powers of arbitrary finite-state maps

Given a native De Morgan circuit of size s computing a map f on n-bit states, every fixed natural exponent T and integer $1 \le L \le 2^n$ admit a circuit for f^T of size at most

$$40000\left(Ls + (n+1)^3\left(L + \left\lfloor \frac{2^n}{L} \right\rfloor + 1\right)\right).$$

The map need not be invertible. The bound is independent of T, but T is fixed when the circuit is chosen; it is not an input to this circuit. Checkpoints are chosen nonconstructively, without a certified preprocessing runtime. [109]

`Cslib.Circuits.Boolean.Iteration.fixed_power`

14.2 An input time for an arbitrary finite-state map

With f, s, n, L as in the fixed-power result, define

$$B = Ls + (n+1)^3\left(L + \left\lfloor \frac{2^n}{L} \right\rfloor + 1\right).$$

For every $m \in \mathbb{N}$, one circuit on $n+m$ inputs computes $f^T(x)$ for all n-bit states x and all m-bit times $0 \le T < 2^m$, using at most

$$m(40000B + 4n)$$

gates. The time is supplied least-significant bit first. This theorem allows noninvertible maps; its linear dependence on the time width differs from the additive sparse-counter bound below. Circuit existence is nonuniform, with no certified checkpoint-construction runtime.[109]

`Cslib.Circuits.Boolean.Iteration.variable_time`

14.3 Batched iteration with a shared table

Let a size-s native De Morgan circuit compute f on n bits. For integers $0 \le r \le 2^n$ and $1 \le L \le 2^n$, put

$$B_r = rLs + (n+1)^3\left(rL + \left\lfloor \frac{2^n}{L} \right\rfloor + 1\right).$$

For every fixed $T \in \mathbb{N}$, one circuit applies f^T to r independent states with at most $100000B_r$ gates. With a separate m-bit input time for each state, a circuit of size at most

$$m(100000B_r + 4rn)$$

computes all r answers. The table term $\lfloor 2^n/L \rfloor$ is charged once in B_r, rather than once per state. All gates, including constants, are charged. These are nonuniform size bounds, not certified preprocessing or circuit-generation times.[109]

`Cslib.Circuits.Boolean.Iteration.batched_fixed_power`
`Cslib.Circuits.Boolean.Iteration.batched_variable_time`

Sparse-counter conventions

Write $W_n = \mathbb{Z}/2^n\mathbb{Z}$ for the n-bit words, represented least-significant bit first, and let $c(x) = x + 1$. For maps on W_n, put

$$D(f, g) = \{x \in W_n : f(x) \neq g(x)\}.$$

Circuit size below counts native De Morgan gates, including constants, comparisons, selection and addition. These are nonuniform existence results: they provide no efficient algorithm for constructing the correction table. The port follows Schlesinger's iterated-functions development; the source credits Moreno and Rivera for the related block characterization of almost-commuting permutations. [90, 109]

14.4 Maps commuting with an edited counter

For any maps $f, h : W_n \to W_n$, if h is injective and $h \circ f = f \circ h$, then there is a circuit computing all n output bits of h with size at most

$$100(n + 1)(|D(f, c)| + 1).$$

There is no assumption that f is a permutation or that a small circuit for f is supplied. The assertion includes $n = 0$.

`Cslib.Circuits.Boolean.Iteration.sparse_counter_centralizer`

14.5 Fixed signed powers and coordinate transfer

For every permutation f of W_n and every integer T, a circuit computes f^T with size at most

$$100(n + 1)(|D(f, c)| + 1).$$

The size bound is independent of the magnitude and sign of T. The quantifiers are "for every T, there exists a circuit": T is fixed, not an additional input to a single circuit.

`Cslib.Circuits.Boolean.Iteration.sparse_counter_power`

Coordinate transfer. Let f, g, e be permutations of W_n such that $e(g(x)) = e(x) + 1$ for every x. Suppose circuits E and E^{-1} compute e and its inverse, respectively. For every $T \in \mathbb{Z}$, there is a circuit for f^T with size at most

$$|E| + |E^{-1}| + 100(n + 1)(|D(f, g)| + 1).$$

Both coordinate conversions are charged. The theorem does not assert that an arbitrary cycle has cheap coordinates or complete the general iterated-function circuit-bound program.

`Cslib.Circuits.Boolean.Iteration.sparse_coordinate_power`

An input iteration count

Keep the notation W_n, c and D introduced above. Here a circuit reads an n-bit initial state x and an m-bit nonnegative integer T, both least-significant bit first. The permutation is fixed when choosing the circuit. The chart and circuit are chosen nonuniformly; no efficient preprocessing or circuit-generation algorithm is asserted.[109]

14.6 Sparse-counter input times and cycle labels

For every permutation f of W_n and every $m \in \mathbb{N}$, there is a native De Morgan circuit with $n+m$ inputs and n outputs computing

$$(x, T) \longmapsto f^T(x), \qquad 0 \le T < 2^m,$$

of size at most

$$1000(n+3)(|D(f,c)| + m + 1).$$

The dependence on the edit count and time width is additive inside the parentheses. One circuit handles all initial states and all m-bit times. The bound includes constants, comparisons and selection gates, and permits $n = 0$ or $m = 0$.
`Cslib.Circuits.Boolean.Iteration.sparse_counter_variable_time`

Canonical coordinates. For every permutation f of W_n, there is a circuit with n inputs and $n + 2(n+2)$ outputs computing the canonical cycle representative, cycle length and phase of the input. Its size is at most

$$500(n+3)(|D(f,c)| + 1).$$

The representative is the numerically least word on the cycle. The phase is the forward distance from that representative, modulo the cycle length. The period and phase each occupy $n+2$ bits, so even a period of 2^n fits.
`Cslib.Circuits.Boolean.Iteration.sparse_counter_labels`

Coordinate transfer. Let f, g, e be permutations of W_n satisfying $e(g(x)) = e(x) + 1$, with supplied circuits E and E^{-1} for e and its inverse. For every time width m, a circuit computing $(x, T) \mapsto f^T(x)$ has size at most

$$|E| + |E^{-1}| + 1000(n+3)(|D(f,g)| + m + 1).$$

The m time bits pass through unchanged to the iteration stage. Both state coordinate conversions are charged once, independently of m.
`Cslib.Circuits.Boolean.Iteration.sparse_coordinate_variable_time`

References

[1] Threshold circuits as decision trees and finite-state programs. Research draft, second pass, 2026. URL https://github.com/SamuelSchlesinger/threshold-circuits/blob/791fa2749317fbcc7b5c5b030a1e6fadd28cc44c/threshold_circuit_representations.tex. October 18, 2026; repository revision 791fa2749317fbcc7b5c5b030a1e6fadd28cc44c.

[2] Leonard Adleman. Two theorems on random polynomial time. In *19th Annual Symposium on Foundations of Computer Science*, pages 75–83. IEEE Computer Society, 1978. doi: 10.1109/SFCS.1978.37.

[3] M. Ajtai, J. Komlós, and E. Szemerédi. Deterministic simulation in LOGSPACE. In *Proceedings of the Nineteenth Annual ACM Symposium on Theory of Computing*, pages 132–140. ACM, 1987. doi: 10.1145/28395.28410. URL https://doi.org/10.1145/28395.28410.

[4] Miklós Ajtai. Σ^1_1-formulae on finite structures. *Annals of Pure and Applied Logic*, 24(1):1–48, 1983. doi: 10.1016/0168-0072(83)90038-6.

[5] Eric W. Allender and Roy S. Rubinstein. P-printable sets. *SIAM Journal on Computing*, 17 (6):1193–1202, 1988. doi: 10.1137/0217075.

[6] Dana Angluin. Queries and concept learning. *Machine Learning*, 2(4):319–342, 1988. doi: 10.1007/BF00116828.

[7] Sanjeev Arora and Boaz Barak. *Computational Complexity - A Modern Approach*. Cambridge University Press, 2009.

[8] Sanjeev Arora, Carsten Lund, Rajeev Motwani, Madhu Sudan, and Mario Szegedy. Proof verification and the hardness of approximation problems. *Journal of the ACM*, 45(3):501–555, 1998. doi: 10.1145/278298.278306.

[9] László Babai. Trading group theory for randomness. In *Proceedings of the 17th Annual ACM Symposium on Theory of Computing*, pages 421–429, 1985. doi: 10.1145/22145.22192.

[10] László Babai and Shlomo Moran. Arthur–Merlin games: A randomized proof system, and a hierarchy of complexity classes. *Journal of Computer and System Sciences*, 36(2):254–276, 1988. doi: 10.1016/0022-0000(88)90028-1.

[11] Theodore Baker, John Gill, and Robert Solovay. Relativizations of the $\mathcal{P} =? \mathcal{NP}$ question. *SIAM Journal on Computing*, 4(4):431–442, 1975. doi: 10.1137/0204037.

[12] Boaz Barak. CS 127: Pseudorandomness, 2016. URL https://www.boazbarak.org/cs127spring16/chap03-pseudorandom-generators. Length extension for pseudorandom generators.

[13] David A. Barrington. Bounded-width polynomial-size branching programs recognize exactly those languages in NC^1. *Journal of Computer and System Sciences*, 38(1):150–164, 1989.

[14] David A. Mix Barrington and Denis Thérien. Finite monoids and the fine structure of NC^1. *Journal of the ACM*, 35(4):941–952, 1988. doi: 10.1145/48014.63138.

[15] Richard Beigel, John Gill, and Ulrich Hertrampf. Counting classes: Thresholds, parity, mods, and fewness. In *STACS 90: 7th Annual Symposium on Theoretical Aspects of Computer Science*, volume 415 of *Lecture Notes in Computer Science*, pages 49–57. Springer, 1990. doi: 10.1007/3-540-52282-4_31.

[16] Eli Ben-Sasson and Avi Wigderson. Short proofs are narrow—resolution made simple. *Journal of the ACM*, 48(2):149–169, 2001. doi: 10.1145/375827.375835. URL https://doi.org/10.1145/375827.375835.

[17] Richard S. Bird. A simple division-free algorithm for computing determinants. *Information Processing Letters*, 111(21–22):1072–1074, 2011. doi: 10.1016/j.ipl.2011.08.006.

[18] Andreas Blass and Yuri Gurevich. On the unique satisfiability problem. *Information and Control*, 55(1–3):80–88, 1982. doi: 10.1016/S0019-9958(82)90439-9.

[19] Elmar Böhler, Christian Glaßer, and Daniel Meister. Error-bounded probabilistic computations between MA and AM. *Journal of Computer and System Sciences*, 72(6): 1043–1076, 2006.

[20] Aline Bonami. Étude des coefficients de Fourier des fonctions de $L^p(G)$. *Annales de l'Institut Fourier*, 20(2):335–402, 1970. doi: 10.5802/aif.357.

[21] Ravi B. Boppana, Johan Håstad, and Stathis Zachos. Does co-NP have short interactive proofs? *Information Processing Letters*, 25(2):127–132, 1987. doi: 10.1016/0020-0190(87)90232-8.

[22] Bernd Borchert, Lane A. Hemaspaandra, and Jörg Rothe. Restrictive acceptance suffices for equivalence problems. *LMS Journal of Computation and Mathematics*, 3:86–95, 2000. doi: 10.1112/S146115700000022X. URL https://arxiv.org/abs/cs/9907041.

[23] Harry Buhrman and Ronald de Wolf. Complexity measures and decision tree complexity: a survey. *Theoretical Computer Science*, 288(1):21–43, 2002.

[24] Jin-Yi Cai and Lane A. Hemachandra. On the power of parity polynomial time. *Mathematical Systems Theory*, 23(1):95–106, 1990. doi: 10.1007/BF02090768.

[25] Jin-Yi Cai, Thomas Gundermann, Juris Hartmanis, Lane A. Hemachandra, Vivian Sewelson, Klaus Wagner, and Gerd Wechsung. The boolean hierarchy I: Structural properties. *SIAM Journal on Computing*, 17(6):1232–1252, 1988. doi: 10.1137/0217078.

[26] Bruno P. Cavalar and Igor C. Oliveira. Boolean circuit complexity and two-dimensional cover problems. Technical Report TR25-033, Electronic Colloquium on Computational Complexity, 2025. URL https://eccc.weizmann.ac.il/report/2025/033/.

[27] Ashok K. Chandra, Dexter C. Kozen, and Larry J. Stockmeyer. Alternation. *Journal of the ACM*, 28(1):114–133, 1981. doi: 10.1145/322234.322243.

[28] Ruiwen Chen and Valentine Kabanets. Correlation bounds and #sat algorithms for small linear-size circuits. In *Computing and Combinatorics (COCOON 2015)*, Lecture Notes in Computer Science, pages 211–222. Springer, 2015. doi: 10.1007/978-3-319-21398-9_17.

[29] Fan R. K. Chung, Ronald L. Graham, Peter Frankl, and James B. Shearer. Some intersection theorems for ordered sets and graphs. *Journal of Combinatorial Theory, Series A*, 43(1): 23–37, 1986. doi: 10.1016/0097-3165(86)90019-1.

[30] James Cook and Ian Mertz. Tree evaluation is in space $O(\log n \cdot \log\log n)$. Technical Report TR23-174, Electronic Colloquium on Computational Complexity, 2023. Conference version in STOC 2024.

[31] Stephen A. Cook and Robert A. Reckhow. The relative efficiency of propositional proof systems. *The Journal of Symbolic Logic*, 44(1):36–50, 1979. doi: 10.2307/2273702.

[32] Marek Cygan, Fedor V. Fomin, Łukasz Kowalik, Daniel Lokshtanov, Dániel Marx, Marcin Pilipczuk, Michał Pilipczuk, and Saket Saurabh. *Parameterized Algorithms*. Springer, Cham, 2015. doi: 10.1007/978-3-319-21275-3.

[33] Irit Dinur. The PCP theorem by gap amplification. *Journal of the ACM*, 54(3):12, 2007. doi: 10.1145/1236457.1236459.

[34] Paul Erdős and Richard Rado. Intersection theorems for systems of sets. *Journal of the London Mathematical Society*, s1-35(1):85–90, 1960. doi: 10.1112/jlms/s1-35.1.85.

[35] Ronald Fagin. Generalized first-order spectra and polynomial-time recognizable sets. In Richard M. Karp, editor, *Complexity of Computation*, volume 7 of *SIAM-AMS Proceedings*, pages 43–73. 1974.

[36] Stephen A. Fenner. PP-lowness and a simple definition of AWPP. *Theory of Computing Systems*, 36(2):199–212, 2003. doi: 10.1007/s00224-002-1089-8. URL https://eccc.weizmann.ac.il/report/2002/036/.

[37] Stephen A. Fenner, Lance J. Fortnow, and Stuart A. Kurtz. Gap-definable counting classes. *Journal of Computer and System Sciences*, 48(1):116–148, 1994. doi: 10.1016/S0022-0000(05)80024-8.

[38] Stephen A. Fenner, Lance J. Fortnow, Stuart A. Kurtz, and Lide Li. An oracle builder's toolkit. *Information and Computation*, 182(2):95–136, 2003. doi: 10.1016/S0890-5401(03)00018-X. URL https://lance.fortnow.com/papers/files/obt.pdf.

[39] Lance Fortnow. Counting complexity. In Lane A. Hemaspaandra and Alan L. Selman, editors, *Complexity Theory Retrospective II*, pages 81–107. Springer, 1997. doi: 10.1007/978-1-4612-1872-2_4. URL https://lance.fortnow.com/papers/files/counting.pdf.

[40] Ehud Friedgut. Boolean functions with low average sensitivity depend on few coordinates. *Combinatorica*, 18(1):27–35, 1998. doi: 10.1007/PL00009809.

[41] Merrick Furst, James B. Saxe, and Michael Sipser. Parity, circuits, and the polynomial-time hierarchy. *Mathematical Systems Theory*, 17(1):13–27, 1984. doi: 10.1007/BF01744431.

[42] Michael R. Garey and David S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, San Francisco, 1979.

[43] John Gill. Computational complexity of probabilistic Turing machines. *SIAM Journal on Computing*, 6(4):675–695, 1977. doi: 10.1137/0206049.

[44] Oded Goldreich. *Foundations of Cryptography, Volume 1: Basic Tools*. Cambridge University Press, 2001. doi: 10.1017/CBO9780511546891.

[45] Oded Goldreich. On the Cook–Mertz tree evaluation procedure. Technical Report TR24-109, Electronic Colloquium on Computational Complexity, 2024. Revision 2, 27 December 2024.

[46] Oded Goldreich and Leonid A. Levin. A hard-core predicate for all one-way functions. In *Proceedings of the Twenty-First Annual ACM Symposium on Theory of Computing*, pages 25–32, 1989. doi: 10.1145/73007.73010. URL https://www.wisdom.weizmann.ac.il/~oded/X/gl.pdf.

[47] Oded Goldreich, Shafi Goldwasser, and Silvio Micali. How to construct random functions. *Journal of the ACM*, 33(4):792–807, 1986. URL https://www.wisdom.weizmann.ac.il/~oded/X/ggm-jacm.pdf.

[48] Oded Goldreich, Silvio Micali, and Avi Wigderson. Proofs that yield nothing but their validity or all languages in NP have zero-knowledge proof systems. *Journal of the ACM*, 38 (3):690–728, 1991. doi: 10.1145/116825.116852.

[49] Leslie M. Goldschlager. The monotone and planar circuit value problems are log space complete for P. *ACM SIGACT News*, 9(2):25–29, 1977. doi: 10.1145/1008354.1008356.

[50] Shafi Goldwasser and Michael Sipser. Private coins versus public coins in interactive proof systems. In *Proceedings of the 18th Annual ACM Symposium on Theory of Computing*, pages 59–68, 1986. doi: 10.1145/12130.12137.

[51] Alexander Golovnev, Alexander S. Kulikov, Alexander V. Smal, and Suguru Tamaki. Circuit size lower bounds and #sat upper bounds through a general framework. In *41st International Symposium on Mathematical Foundations of Computer Science (MFCS 2016)*, volume 58 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 45:1–45:16. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2016. doi: 10.4230/LIPIcs.MFCS.2016.45.

[52] Venkatesan Guruswami, Atri Rudra, and Madhu Sudan. Essential coding theory, 2019. URL https://cse.buffalo.edu/faculty/atri/courses/coding-theory/book/. Draft book, 2019.

[53] Johan Håstad. Almost optimal lower bounds for small depth circuits. In *Proceedings of the 18th Annual ACM Symposium on Theory of Computing*, pages 6–20, 1986. doi: 10.1145/12130.12132.

[54] Edith Hemaspaandra, Lane A. Hemaspaandra, Holger Spakowski, and Osamu Watanabe. The robustness of LWPP and WPP, with an application to graph reconstruction. In *43rd International Symposium on Mathematical Foundations of Computer Science*, volume 117 of *Leibniz International Proceedings in Informatics*, pages 51:1–51:14. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2018. doi: 10.4230/LIPIcs.MFCS.2018.51.

[55] Wassily Hoeffding. Probability inequalities for sums of bounded random variables. *Journal of the American Statistical Association*, 58(301):13–30, 1963. doi: 10.1080/01621459.1963.10500830.

[56] Thomas Holenstein. Pseudorandom generators from one-way functions: A simple construction for any hardness. In *Theory of Cryptography (TCC 2006)*, 2006. URL https://crypto.ethz.ch/publications/files/Holens06.pdf.

[57] Shlomo Hoory, Nathan Linial, and Avi Wigderson. Expander graphs and their applications. *Bulletin of the American Mathematical Society*, 43(4):439–561, 2006. doi: 10.1090/S0273-0979-06-01126-8.

[58] Hao Huang. Induced subgraphs of hypercubes and a proof of the sensitivity conjecture. *Annals of Mathematics*, 190(3):949–955, 2019. doi: 10.4007/annals.2019.190.3.6.

[59] Johan Håstad, Russell Impagliazzo, Leonid A. Levin, and Michael Luby. A pseudorandom generator from any one-way function. *SIAM Journal on Computing*, 28(4):1364–1396, 1999. doi: 10.1137/S0097539793244708. URL https://doi.org/10.1137/S0097539793244708.

[60] Neil Immerman. *Descriptive Complexity*. Graduate Texts in Computer Science. Springer, 1999. doi: 10.1007/978-1-4612-0539-5.

[61] Neil Immerman and Susan Landau. The complexity of iterated multiplication. *Information and Computation*, 116(1):103–116, 1995. URL https://people.cs.umass.edu/~immerman/pub/mult.pdf.

[62] Russell Impagliazzo and Avi Wigderson. P = BPP if E requires exponential circuits: Derandomizing the XOR lemma. In *Proceedings of the 29th Annual ACM Symposium on Theory of Computing*, STOC 1997, pages 220–229, 1997. doi: 10.1145/258533.258590.

[63] Russell Impagliazzo, Ramamohan Paturi, and Francis Zane. Which problems have strongly exponential complexity? *Journal of Computer and System Sciences*, 63(4):512–530, 2001. doi: 10.1006/jcss.2001.1774.

[64] Stasys Jukna. *Extremal Combinatorics: With Applications in Computer Science*. Texts in Theoretical Computer Science. An EATCS Series. Springer, 2 edition, 2011. doi: 10.1007/978-3-642-17364-6.

[65] Stasys Jukna. *Boolean Function Complexity: Advances and Frontiers*, volume 27 of *Algorithms and Combinatorics*. Springer, 2012. doi: 10.1007/978-3-642-24508-4. URL https://web.vu.lt/mif/s.jukna/boolean/index.html.

[66] Valentine Kabanets and Jin-Yi Cai. Circuit minimization problem. In *Proceedings of the Thirty-Second Annual ACM Symposium on Theory of Computing (STOC 2000)*, pages 73–79, 2000. doi: 10.1145/335305.335314.

[67] Nabil Kahale. Eigenvalues and expansion of regular graphs. *Journal of the ACM*, 42(5): 1091–1106, 1995. doi: 10.1145/210118.210136. URL https://doi.org/10.1145/210118.210136.

[68] Jeff Kahn, Gil Kalai, and Nathan Linial. The influence of variables on Boolean functions. In *29th Annual Symposium on Foundations of Computer Science*, pages 68–80. IEEE Computer Society, 1988. doi: 10.1109/SFCS.1988.21923.

[69] Bala Kalyanasundaram and Georg Schnitger. The probabilistic communication complexity of set intersection. *SIAM Journal on Discrete Mathematics*, 5(4):545–557, 1992. doi: 10.1137/0405044.

[70] Mauricio Karchmer and Avi Wigderson. Monotone circuits for connectivity require super-logarithmic depth. *SIAM Journal on Discrete Mathematics*, 3(2):255–265, 1990.

[71] Richard M. Karp. Reducibility among combinatorial problems. In Raymond E. Miller and James W. Thatcher, editors, *Complexity of Computer Computations*, pages 85–103. Plenum Press, 1972. doi: 10.1007/978-1-4684-2001-2_9.

[72] Richard M. Karp and Richard J. Lipton. Some connections between nonuniform and uniform complexity classes. In *Proceedings of the 12th Annual ACM Symposium on Theory of Computing*, STOC 1980, pages 302–309, 1980. doi: 10.1145/800141.804678.

[73] Jonathan Katz and Yehuda Lindell. *Introduction to Modern Cryptography*. CRC Press, 3rd edition, 2020. ISBN 9780815354369.

[74] V. M. Khrapchenko. Complexity of the realization of a linear function in the class of π-circuits. *Mathematical Notes of the Academy of Sciences of the USSR*, 9:21–23, 1971. Russian original: Matematicheskie Zametki 9(1), 35–40.

[75] V. M. Khrapchenko. Method of determining lower bounds for the complexity of π-schemes. *Mathematical Notes of the Academy of Sciences of the USSR*, 10:474–479, 1971. doi: 10.1007/BF01747074. Russian original: Matematicheskie Zametki 10(1), 83–92.

[76] Johannes Köbler, Uwe Schöning, Seinosuke Toda, and Jacobo Torán. Turing machines with few accepting computations and low sets for PP. *Journal of Computer and System Sciences*, 44(2):272–286, 1992. doi: 10.1016/0022-0000(92)90022-B.

[77] Eyal Kushilevitz and Yishay Mansour. Learning decision trees using the Fourier spectrum. *SIAM Journal on Computing*, 22(6):1331–1348, 1993. doi: 10.1137/0222080.

[78] Eyal Kushilevitz and Noam Nisan. *Communication Complexity*. Cambridge University Press, 1997.

[79] Richard E. Ladner. The circuit value problem is log space complete for P. *ACM SIGACT News*, 7(1):18–20, 1975. doi: 10.1145/990518.990519.

[80] Clemens Lautemann. BPP and the polynomial hierarchy. *Information Processing Letters*, 17 (4):215–217, 1983. doi: 10.1016/0020-0190(83)90044-3.

[81] Victor Lecomte and Prasanna Ramakrishnan. Optimal shallow circuits for majority, 2026.

[82] Harry R. Lewis and Christos H. Papadimitriou. Symmetric space-bounded computation. *Theoretical Computer Science*, 19(2):161–187, 1982. doi: 10.1016/0304-3975(82)90058-5.

[83] Ming Li and Paul Vitányi. *An Introduction to Kolmogorov Complexity and Its Applications*. Texts in Computer Science. Springer, Cham, 4th edition, 2019. doi: 10.1007/978-3-030-11298-1.

[84] David Lichtenstein and Michael Sipser. GO is polynomial-space hard. *Journal of the ACM*, 27(2):393–401, 1980. doi: 10.1145/322186.322201.

[85] Nathan Linial, Yishay Mansour, and Noam Nisan. Constant depth circuits, Fourier transform, and learnability. *Journal of the ACM*, 40(3):607–620, 1993. doi: 10.1145/174130.174138.

[86] Daniel Lokshtanov, Dániel Marx, and Saket Saurabh. Lower bounds based on the Exponential Time Hypothesis. *Bulletin of the EATCS*, 105:41–72, 2011.

[87] Lynn H. Loomis and Hassler Whitney. An inequality related to the isoperimetric inequality. *Bulletin of the American Mathematical Society*, 55(10):961–962, 1949. doi: 10.1090/S0002-9904-1949-09320-5.

[88] Carsten Lund, Lance Fortnow, Howard Karloff, and Noam Nisan. Algebraic methods for interactive proof systems. *Journal of the ACM*, 39(4):859–868, 1992. doi: 10.1145/146585.146605.

[89] Oleg B. Lupanov. On a method of circuit synthesis. *Izvestiya Vysshikh Uchebnykh Zavedenii. Radiofizika*, 1(1):120–140, 1958. URL https://radiophysics.unn.ru/sites/default/files/papers/1958_1_120.pdf. In Russian.

[90] Rutilo Moreno and Luis Manuel Rivera. Blocks in cycles and k-commuting permutations. *SpringerPlus*, 5:1949, 2016. doi: 10.1186/s40064-016-3638-7. URL https://doi.org/10.1186/s40064-016-3638-7. Article 1949.

[91] Ilan Newman. Private vs. common random bits in communication complexity. *Information Processing Letters*, 39(2):67–71, 1991. doi: 10.1016/0020-0190(91)90157-D.

[92] Noam Nisan. CREW PRAMs and decision trees. *SIAM Journal on Computing*, 20(6): 999–1007, 1991.

[93] Noam Nisan and Mario Szegedy. On the degree of Boolean functions as real polynomials. *Computational Complexity*, 4(4):301–313, 1994. doi: 10.1007/BF01263419.

[94] Noam Nisan and Avi Wigderson. Hardness vs randomness. *Journal of Computer and System Sciences*, 49(2):149–167, 1994. doi: 10.1016/S0022-0000(05)80043-1.

[95] Ryan O'Donnell. *Analysis of Boolean Functions*. Cambridge University Press, 2014. doi: 10.1017/CBO9781139814782.

[96] Igor Carboni Oliveira. Notes on the method of approximations and the emergence of the fusion method, August 2018. URL https://www.dcs.warwick.ac.uk/~igorcarb/documents/papers/approximation-method-fusion-note.pdf. Expository notes dated August 31, 2018.

[97] OpenAI. Sensitivity versus block sensitivity: a superquadratic separation, 2026. URL https://github.com/openai/math/tree/adc7f1241b42e322a6451854ab7e4b4c146bf78a/lean/OAI/Combinatorics/Sensitivity. Lean formalization, revision adc7f1241b42e322a6451854ab7e4b4c146bf78a.

[98] Christos H. Papadimitriou. *Computational Complexity*. Addison-Wesley, Reading, Massachusetts, 1994.

[99] Christos H. Papadimitriou and Stathis K. Zachos. Two remarks on the power of counting. In Armin B. Cremers and Hans-Peter Kriegel, editors, *Theoretical Computer Science: 6th GI-Conference, Dortmund, January 5–7, 1983*, volume 145 of *Lecture Notes in Computer Science*, pages 269–275. Springer-Verlag, 1982. doi: 10.1007/BFb0009651. URL https://doi.org/10.1007/BFb0009651. Proceedings published in December 1982 for the January 1983 conference.

[100] Ran Raz and Avi Wigderson. Monotone circuits for matching require linear depth. *Journal of the ACM*, 39(3):736–744, 1992. doi: 10.1145/146637.146684.

[101] Alexander A. Razborov. Lower bounds on the monotone complexity of some Boolean functions. *Soviet Mathematics Doklady*, 31:354–357, 1985. Russian original: Doklady Akademii Nauk SSSR 281(4), 798–801.

[102] Alexander A. Razborov. Lower bounds on the size of bounded depth circuits over a complete basis with logical addition. *Mathematical Notes of the Academy of Sciences of the USSR*, 41 (4):333–338, 1987. doi: 10.1007/BF01137685.

[103] Alexander A. Razborov. On the method of approximations. In *Proceedings of the Twenty-First Annual ACM Symposium on Theory of Computing*, pages 167–176, 1989.

[104] Alexander A. Razborov. On the distributional complexity of disjointness. *Theoretical Computer Science*, 106(2):385–390, 1992. doi: 10.1016/0304-3975(92)90260-M.

[105] Alexander A. Razborov and Steven Rudich. Natural proofs. *Journal of Computer and System Sciences*, 55(1):24–35, 1997. doi: 10.1006/jcss.1997.1494.

[106] Omer Reingold. Undirected connectivity in log-space. *Journal of the ACM*, 55(4):17:1–17:24, 2008. doi: 10.1145/1391289.1391291.

[107] Thomas J. Schaefer. On the complexity of some two-person perfect-information games. *Journal of Computer and System Sciences*, 16(2):185–225, 1978. doi: 10.1016/0022-0000(78)90045-4.

[108] Samuel Schlesinger. Algebraic circuit formalizations in Complexitylib, 2026. URL https://github.com/SamuelSchlesinger/complexitylib/tree/2ad6cb481b70ee6e65e078dffb76b9856be199d3/Complexitylib/Algebraic. Source of ported Lean proofs, revision 2ad6cb481b70ee6e65e078dffb76b9856be199d3.

[109] Samuel Schlesinger. Square-root circuit bounds for iterated functions, 2026. URL https://github.com/SamuelSchlesinger/iterating-functions/tree/c4cfa198a241b3d2cc3bfa5f744a68a66f49be8b/lean. Source of ported Lean proofs, revision c4cfa198a241b3d2cc3bfa5f744a68a66f49be8b.

[110] Samuel Schlesinger. Boolean circuit mass production formalization, 2026. URL https://github.com/SamuelSchlesinger/algebraic-circuits/tree/3998dc48823f7d2798aa814f2cbfbf3ceba146fa/Algebraic. Source of ported Lean proofs, revision 3998dc48823f7d2798aa814f2cbfbf3ceba146fa.

[111] Samuel Schlesinger. Canonical numerical predicates in Complexitylib, 2026. URL https://github.com/SamuelSchlesinger/complexitylib/blob/e5ca965688f1b57f6df15482e895b66452955a40/Complexitylib/DescriptiveComplexity/Numerical.lean. Earlier canonical-expansion interfaces; revision e5ca965688f1b57f6df15482e895b66452955a40.

[112] Samuel Schlesinger. Sensitivity separation in Complexitylib, 2026. URL https://github.com/SamuelSchlesinger/complexitylib/tree/add0aca8ee0d71e68969f2cb45cf91554a50f787/Complexitylib/BooleanAnalysis/Sensitivity. Source of ported Lean proofs, revision add0aca8ee0d71e68969f2cb45cf91554a50f787.

[113] Adi Shamir. IP = PSPACE. *Journal of the ACM*, 39(4):869–877, 1992. doi: 10.1145/146585.146609.

[114] Claude E. Shannon. The synthesis of two-terminal switching circuits. *Bell System Technical Journal*, 28(1):59–98, 1949. doi: 10.1002/j.1538-7305.1949.tb03624.x. URL https://deeplearning.cs.cmu.edu/F24/document/readings/Shannon49.pdf.

[115] A. Shen. IP = SPACE: Simplified proof. *Journal of the ACM*, 39(4):878–880, 1992. doi: 10.1145/146585.146613.

[116] Janos Simon. *On Some Central Problems in Computational Complexity*. PhD thesis, Cornell University, 1975.

[117] Michael Sipser. A complexity theoretic approach to randomness. In *Proceedings of the 15th Annual ACM Symposium on Theory of Computing*, pages 330–335, 1983. doi: 10.1145/800061.808762. URL https://doi.org/10.1145/800061.808762.

[118] Michael Sipser. *Introduction to the Theory of Computation*. Cengage Learning, 3rd edition, 2013.

[119] Roman Smolensky. Algebraic methods in the theory of lower bounds for Boolean circuit complexity. In *Proceedings of the 19th Annual ACM Symposium on Theory of Computing*, pages 77–82, 1987. doi: 10.1145/28395.28404.

[120] Philip M. Spira. On time-hardware complexity tradeoffs for Boolean functions. In *Proceedings of the Fourth Hawaii International Symposium on System Sciences*, pages 525–527, 1971.

[121] Madhu Sudan. Decoding of Reed Solomon codes beyond the error-correction bound. *Journal of Complexity*, 13(1):180–193, 1997. doi: 10.1006/jcom.1997.0439.

[122] Madhu Sudan, Luca Trevisan, and Salil Vadhan. Pseudorandom generators without the XOR lemma. *Journal of Computer and System Sciences*, 62(2):236–266, 2001. doi: 10.1006/jcss.2000.1730.

[123] Michel Talagrand. On Russo's approximate zero-one law. *The Annals of Probability*, 22(3): 1576–1587, 1994. doi: 10.1214/aop/1176988612.

[124] Seinosuke Toda. PP is as hard as the polynomial-time hierarchy. *SIAM Journal on Computing*, 20(5):865–877, 1991. doi: 10.1137/0220053. URL https://doi.org/10.1137/0220053.

[125] G. S. Tseitin. On the complexity of derivation in propositional calculus. In Jörg H. Siekmann and Graham Wrightson, editors, *Automation of Reasoning: 2: Classical Papers on Computational Logic 1967–1970*, pages 466–483. Springer, 1983. doi: 10.1007/978-3-642-81955-1_28. URL https://doi.org/10.1007/978-3-642-81955-1_28. Originally published in Russian in 1968.

[126] Alasdair Urquhart. Hard examples for resolution. *Journal of the ACM*, 34(1):209–219, 1987. doi: 10.1145/7531.8928. URL https://doi.org/10.1145/7531.8928.

[127] Salil P. Vadhan. Pseudorandomness. *Foundations and Trends in Theoretical Computer Science*, 7(1–3):1–336, 2012. doi: 10.1561/0400000010.

[128] L. G. Valiant. A theory of the learnable. In *Proceedings of the Sixteenth Annual ACM Symposium on Theory of Computing*, STOC '84, pages 436–445, New York, NY, USA, 1984. Association for Computing Machinery. ISBN 0-89791-133-4. doi: 10.1145/800057.808710. URL https://doi.org/10.1145/800057.808710.

[129] Leslie G. Valiant. The complexity of computing the permanent. *Theoretical Computer Science*, 8(2):189–201, 1979. doi: 10.1016/0304-3975(79)90044-6.

[130] Leslie G. Valiant. Completeness classes in algebra. In *Proceedings of the Eleventh Annual ACM Symposium on Theory of Computing*, STOC '79, pages 249–261. ACM, 1979. doi: 10.1145/800135.804419.

[131] Leslie G. Valiant and Vijay V. Vazirani. NP is as easy as detecting unique solutions. *Theoretical Computer Science*, 47:85–93, 1986. doi: 10.1016/0304-3975(86)90135-0.

[132] Klaus W. Wagner. The complexity of combinatorial problems with succinct input representation. *Acta Informatica*, 23(3):325–356, 1986. doi: 10.1007/BF00289117.

[133] Thomas Watson. The complexity of estimating min-entropy. Technical Report TR12-070, Electronic Colloquium on Computational Complexity, 2012. URL https://eccc.weizmann.ac.il/report/2012/070/.

[134] Lloyd R. Welch and Elwyn R. Berlekamp. Error correction for algebraic block codes, 1986. U.S. Patent 4,633,470.

[135] S. V. Yablonskii. Realization of a linear function in the class of π-schemes. *Doklady Akademii Nauk SSSR*, 94(5):805–806, 1954. In Russian.

[136] Andrew Chi-Chih Yao. Probabilistic computations: Toward a unified measure of complexity. In *Proceedings of the 18th Annual Symposium on Foundations of Computer Science*, FOCS 1977, pages 222–227, 1977. doi: 10.1109/SFCS.1977.24.